DIGITAL LOGIC AND COMPUTER ORGANIZATION

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

LENDI INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institute, Approved by AICTE & Permanently Affiliated to JNTU-GV, Vizianagaram)

(Accredited By NAAC with A Grade and Accredited by NBA)

Jonnada (Village), Denkada (Mandal), Vizianagaram District – 535 005

Phone No. 08922-241111, 241112

E-Mail: lendi_2008@yahoo.com website: www.lendi.org

UNIT – II

Combinational circuits-II: Code Converters, Encoders, Decoders, Multiplexers, De-multiplexers.

Sequential logic circuits: Flip-Flops (SR, JK, T, D), Excitation tables, conversion of Flip Flops, Synchronous and Asynchronous counters, Up/Down counters, Modulus Counters Registers: Bi-directional, Universal Shift Register.

INDEX

S.No	Name of the topic	Page Number
1	**Combinational circuits-II:** Code Converters	3
2	Decoders	12
3	De-multiplexers.	18
4	Encoders	22
5	Multiplexers	26
6	**Sequential logic circuits**	36
7	Flip-Flops (SR, JK, T, D) and Excitation tables	39
8	Conversion of Flip Flops	46
9	Synchronous and Asynchronous counters	48
10	Asynchronous counters (Up/Down counters, Modulus Counters)	49
11	Synchronous counters (Up/Down counters, Modulus Counters)	55
12	Bi-directional Register	64
13	Universal Shift Register	65

UNIT – II
Combinational circuits-II

CODE CONVERTERS

The availability of a large variety of codes for the same discrete elements of information results in the use of different codes by different digital systems. It is sometimes necessary to use the output of one system as the input to another. A conversion circuit must be inserted between the two systems if each uses different codes for the same information. Thus, **a code converter is a circuit that makes the two systems compatible even though each uses a different binary code.**

To convert from binary code A to binary code B, the input lines must supply the bit combination of elements as specified by code A and the output lines must generate the corresponding bit combination of code B. A combinational circuit performs this transformation by means of logic gates.

Example: Design and implement Binary Coded Decimal (BCD) to Excess-3 converter
.**Solution:**

The design procedure that converts binary coded decimal (BCD) to the excess-3 code for the decimal digits is illustrated here. Since each code uses four bits to represent a decimal digit, there must be four input variables and four output variables. We designate the four input binary variables by the symbols *A, B, C,* and *D*, and the four output variables by *w, x, y* , and *z* . The truth table relating the input and output variables is shown below. Note that four binary variables may have 16 bit combinations, but only 10 are listed in the truth table. The six bit combinations not listed for the input variables are don't-care combinations. These values have no meaning in BCD and we assume that they will never occur in actual operation of the circuit. Therefore, we are at liberty to assign to the output variables either a 1 or a 0, whichever gives a simpler circuit.

Table: Truth table for BCD to Excess-3 converter

Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

The maps below are plotted to obtain simplified Boolean functions for the outputs. Each one of the four maps represents one of the four outputs of the circuit as a function of the four input variables. The 1's marked inside the squares are obtained from the minterms that make the output equal to 1. The 1's are obtained from the truth table by going over the output columns one at a time. For example, the column under output *z* has five 1's; therefore, the map for *z* has five 1's, each being in a square corresponding to the minterm that makes *z* equal to 1. The six

don't-care minterms 10 through 15 are marked with an *X*. One possible way to simplify the functions into sum-of-products form is listed under the map of each variable.

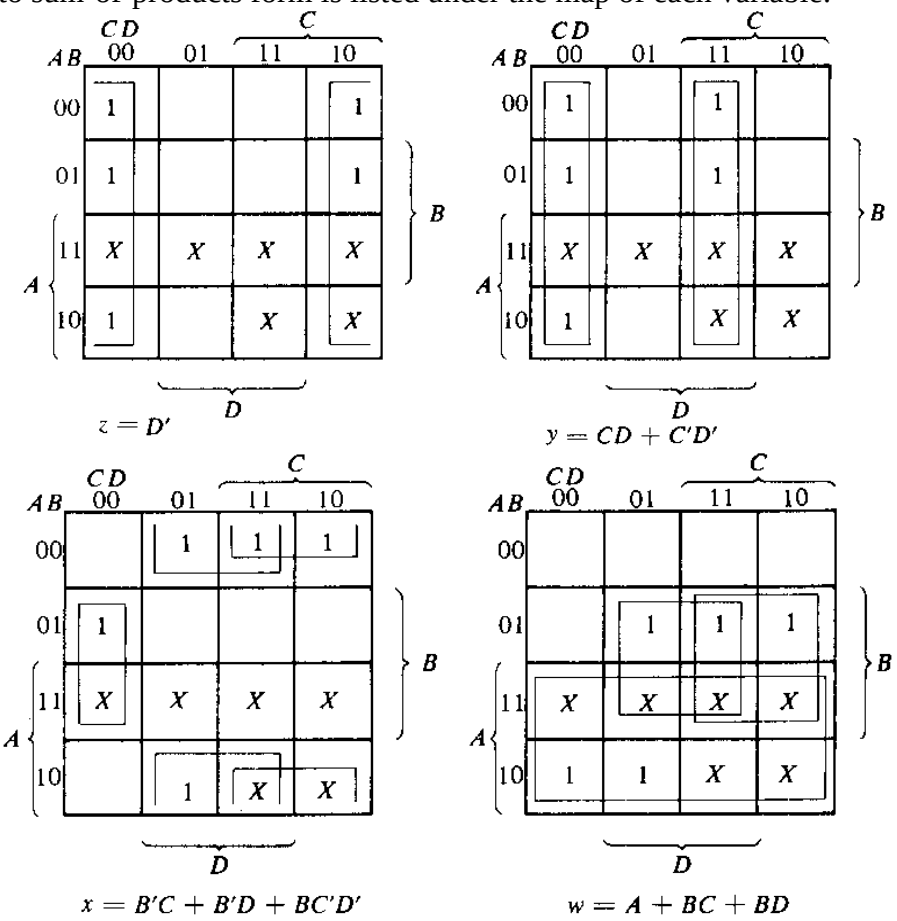


Figure: Maps for BCD to Excess-3 converter

A two-level logic diagram for each output may be obtained directly from the Boolean expressions derived from the maps. There are various other possibilities for a logic diagram that implements this circuit. The expressions obtained above may be manipulated algebraically for the purpose of using common gates for two or more outputs. This manipulation, shown next, illustrates the flexibility obtained with multiple-output systems when implemented with three or more levels of gates:

z = D'
y = CD + C'D' = CD + (C + D)'
x = B'C + B'D + BC'D' = B'(C + D) + BC'D'
= B'(C + D) + B(C + D)'
w = A + BC + BD = A + B(C + D)

The logic diagram that implements these expressions is shown below. Note that the OR gate whose output is *C* + *D* has been used to implement partially each of three outputs. Not counting input inverters, the implementation in sum-of-products form requires seven AND gates and three OR gates. The implementation of logic diagram requires four AND gates, four OR gates, and one inverter. If only the normal inputs are available, the first implementation will require inverters for variables *B*, *C*, and *D*, and the second implementation will require inverters for variables *B* and *D*. Thus, the three-level logic circuit requires fewer gates, all of which in turn require no more than two inputs.

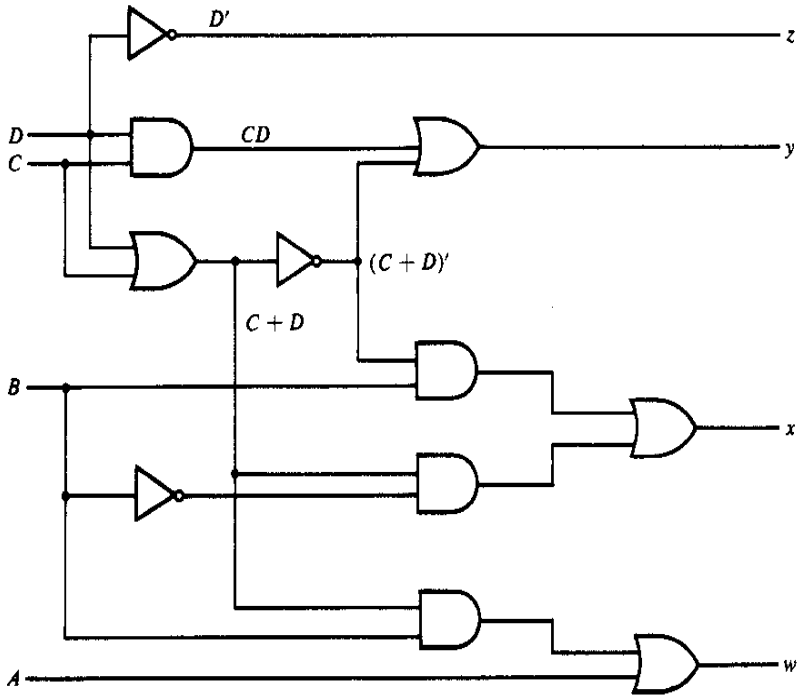


Figure: Logic diagram for BCD to Excess-3 converter

Example: Design and implement 4 bit binary to Gray converter.
Solution:

4-bit binary				4-bit Gray			
B_4	B_3	B_2	B_1	G_4	G_3	G_2	G_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

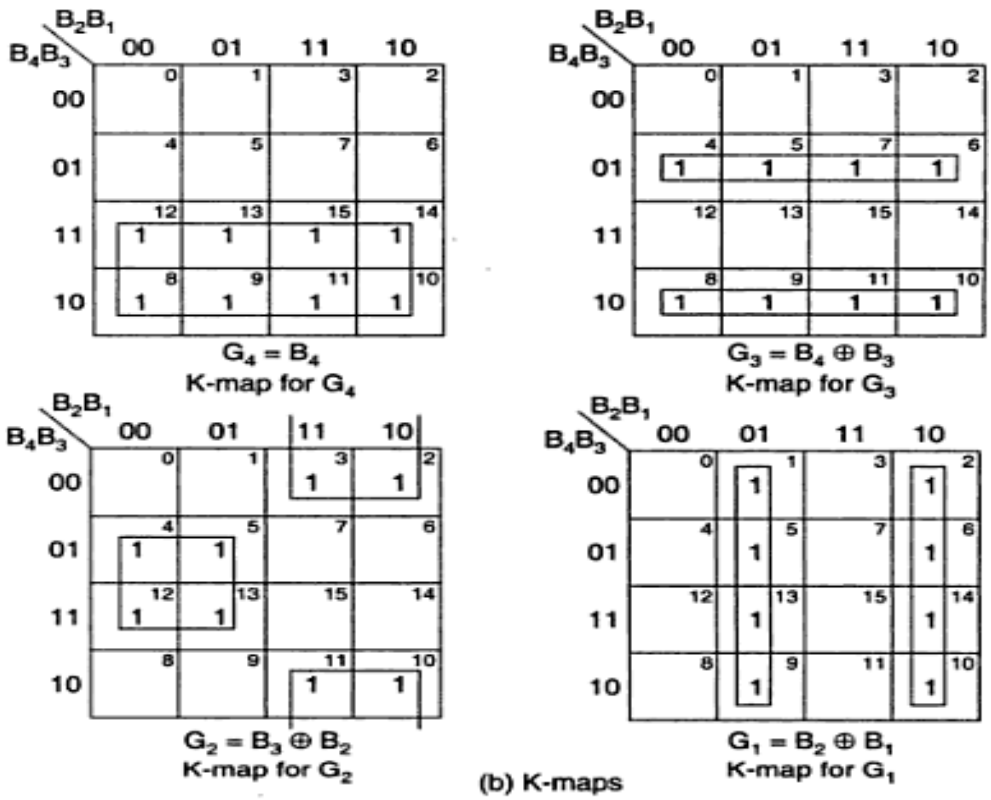
(a) Conversion table

$G_4 = \Sigma m(8, 9, 10, 11, 12, 13, 14, 15)$

$G_3 = \Sigma m(4, 5, 6, 7, 8, 9, 10, 11)$

$G_2 = \Sigma m(2, 3, 4, 5, 10, 11, 12, 13)$

$G_1 = \Sigma m(1, 2, 5, 6, 9, 10, 13, 14)$

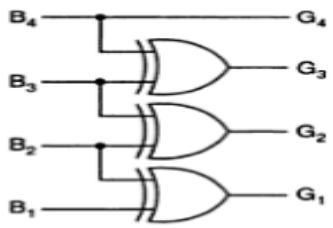


(b) K-maps

4-bit binary-to-Gray code converter.

$$G_4 = B_4$$
$$G_3 = \overline{B}_4 B_3 + B_4 \overline{B}_3 = B_4 \oplus B_3$$
$$G_2 = \overline{B}_3 B_2 + B_3 \overline{B}_2 = B_3 \oplus B_2$$
$$G_1 = \overline{B}_2 B_1 + B_2 \overline{B}_1 = B_2 \oplus B_1$$

Logic Equations



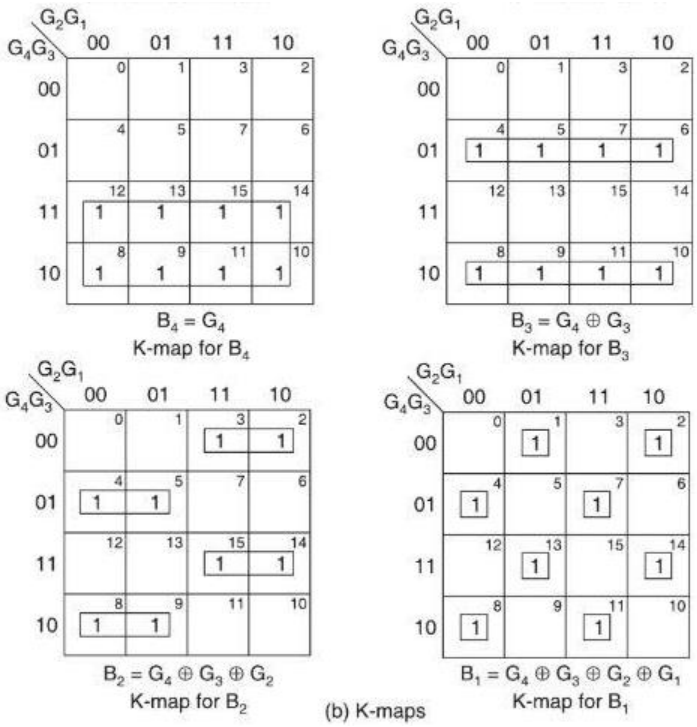
Logic diagram

Example: Design and implement a 4-bit Gray to Binary code converter.
Solution:

4-bit Gray				4-bit binary			
G_4	G_3	G_2	G_1	B_4	B_3	B_2	B_1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

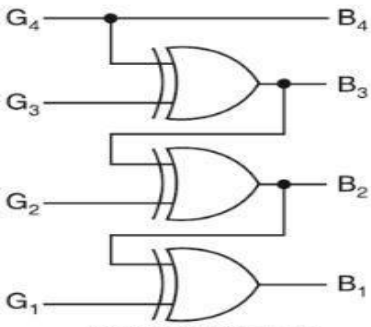
(a) Conversion table

$$B_4 = \Sigma\, m(12, 13, 15, 14, 10, 11, 9, 8) = \Sigma\, m(8, 9, 10, 11, 12, 13, 14, 15)$$
$$B_3 = \Sigma\, m(6, 7, 5, 4, 10, 11, 9, 8) = \Sigma\, m(4, 5, 6, 7, 8, 9, 10, 11)$$
$$B_2 = \Sigma\, m(3, 2, 5, 4, 15, 14, 9, 8) = \Sigma\, m(2, 3, 4, 5, 8, 9, 14, 15)$$
$$B_1 = \Sigma\, m(1, 2, 7, 4, 13, 14, 11, 8) = \Sigma\, m(1, 2, 4, 7, 8, 11, 13, 14)$$



(b) K-maps

$$B_4 = G_4$$
$$B_3 = \bar{G}_4 G_3 + G_4 \bar{G}_3 = G_4 \oplus G_3$$
$$B_2 = \bar{G}_4 G_3 \bar{G}_2 + \bar{G}_4 \bar{G}_3 G_2 + G_4 \bar{G}_3 \bar{G}_2 + G_4 G_3 G_2$$
$$= \bar{G}_4 (G_3 \oplus G_2) + G_4 (\overline{G_3 \oplus G_2}) = G_4 \oplus G_3 \oplus G_2 = B_3 \oplus G_2$$
$$B_1 = \bar{G}_4 \bar{G}_3 \bar{G}_2 G_1 + \bar{G}_4 \bar{G}_3 G_2 \bar{G}_1 + \bar{G}_4 G_3 G_2 G_1 + \bar{G}_4 G_3 \bar{G}_2 \bar{G}_1 + G_4 G_3 \bar{G}_2 G_1$$
$$+ G_4 G_3 G_2 \bar{G}_1 + G_4 \bar{G}_3 G_2 G_1 + G_4 \bar{G}_3 \bar{G}_2 \bar{G}_1$$
$$= \bar{G}_4 \bar{G}_3 (G_2 \oplus G_1) + G_4 G_3 (G_2 \oplus G_1) + \bar{G}_4 G_3 (\overline{G_2 \oplus G_1}) + G_4 \bar{G}_3 (\overline{G_2 \oplus G_1})$$
$$= (G_2 \oplus G_1)(\overline{G_4 \oplus G_3}) + (\overline{G_2 \oplus G_1})(G_4 \oplus G_3)$$
$$= G_4 \oplus G_3 \oplus G_2 \oplus G_1$$
$$= B_2 \oplus G_1$$



(c) Logic diagram

Example: Design and implement a 4-bit Binary to BCD code converter.
Solution:

Decimal	4-bit binary				BCD output				
	B_4	B_3	B_2	B_1	A	B	C	D	E
0	0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	1
2	0	0	1	0	0	0	0	1	0
3	0	0	1	1	0	0	0	1	1
4	0	1	0	0	0	0	1	0	0
5	0	1	0	1	0	0	1	0	1
6	0	1	1	0	0	0	1	1	0
7	0	1	1	1	0	0	1	1	1
8	1	0	0	0	0	1	0	0	0
9	1	0	0	1	0	1	0	0	1
10	1	0	1	0	1	0	0	0	0
11	1	0	1	1	1	0	0	0	1
12	1	1	0	0	1	0	0	1	0
13	1	1	0	1	1	0	0	1	1
14	1	1	1	0	1	0	1	0	0
15	1	1	1	1	1	0	1	0	1

(a) Conversion table

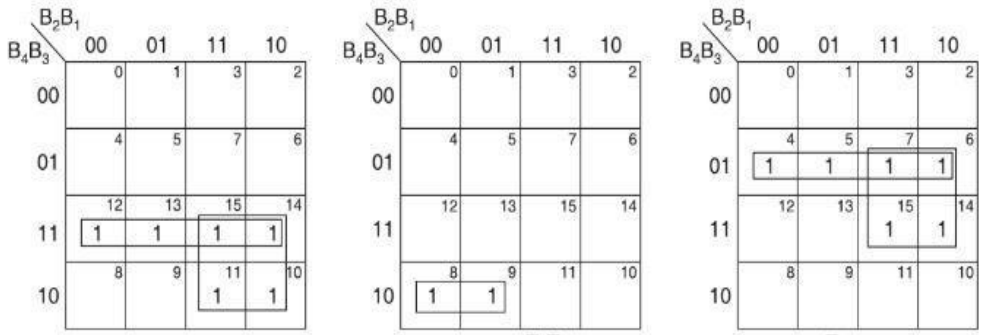
$A = \Sigma\, m(10, 11, 12, 13, 14, 15)$

$B = \Sigma\, m(8, 9)$

$C = \Sigma\, m(4, 5, 6, 7, 14, 15)$

$D = \Sigma\, m(2, 3, 6, 7, 12, 13)$

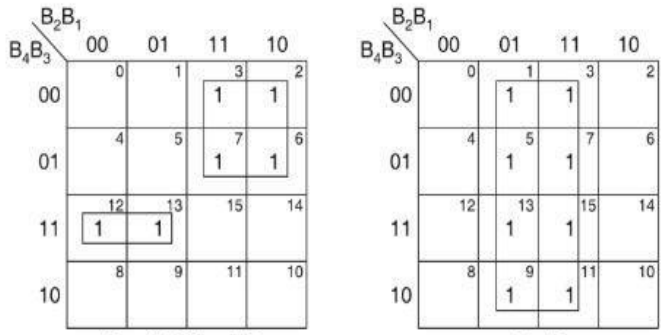
$E = \Sigma\, m(1, 3, 5, 7, 9, 11, 13, 15)$



$A = B_4B_3 + B_4B_2$
K-map for A

$B = B_4\bar{B}_3\bar{B}_2$
K-map for B

$C = \bar{B}_4B_3 + B_3B_2$
K-map for C



$D = B_4B_3\bar{B}_2 + \bar{B}_4B_2$
K-map for D

$E = B_1$
K-map for E

(c) K-maps

$A = B_4B_3 + B_4B_2$

$B = B_4\bar{B}_3\bar{B}_2$

$C = \bar{B}_4B_3 + B_3B_2$

$D = B_4B_3\bar{B}_2 + \bar{B}_4B_2$

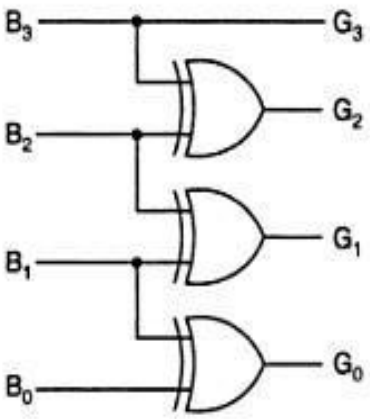
$E = B_1$

Block diagram

Example: Design and implement a 4-bit BCD to Gray code converter.
Solution:

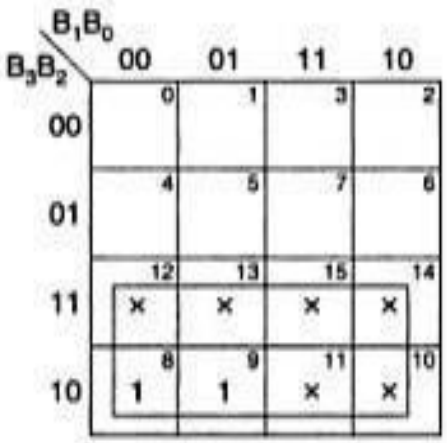
BCD code				Gray code			
B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1

(a) BCD-to-Gray code conversion table

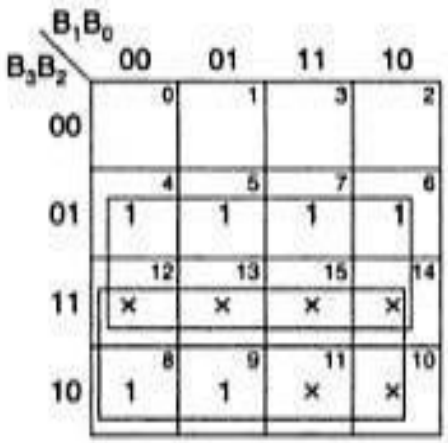


(b) Logic diagram

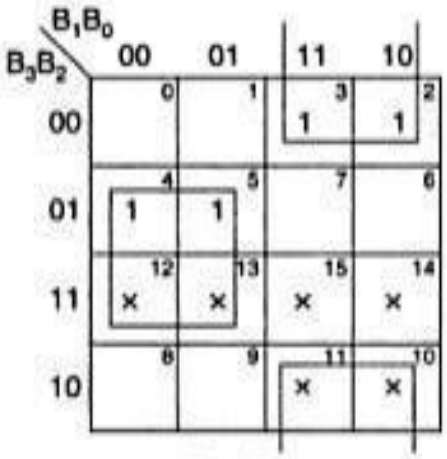
BCD-to-Gray code converter.



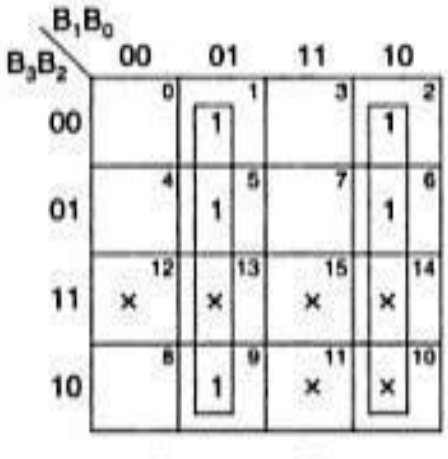
$G_3 = B_3$



$G_2 = B_2 + B_3$



$G_1 = B_2\bar{B}_1 + \bar{B}_2B_1 = B_2 \oplus B_1$



$G_0 = \bar{B}_1B_0 + B_1 . \bar{B}_0 = B_1 \oplus B_0$

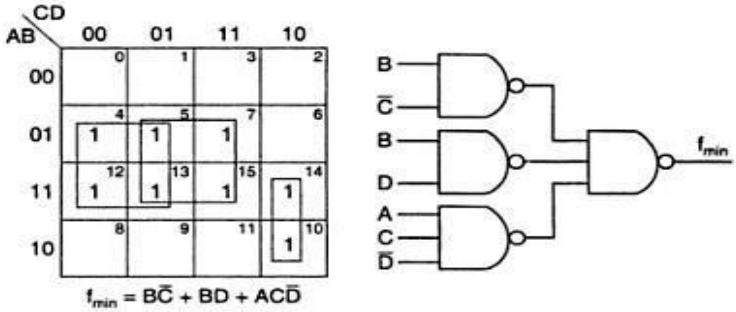
K-maps for a BCD-to-Gray code converter.

Example: Design of a SOP circuit to detect the decimal numbers 5 through 12 in a 4-bit Gray code input.

Solution:

Decimal number	4-bit Gray code A	B	C	D	Output f
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	1	0
3	0	0	1	0	0
4	0	1	1	0	0
5	0	1	1	1	1
6	0	1	0	1	1
7	0	1	0	0	1
8	1	1	0	0	1
9	1	1	0	1	1
10	1	1	1	1	1
11	1	1	1	0	1
12	1	0	1	0	1
13	1	0	1	1	0
14	1	0	0	1	0
15	1	0	0	0	0

(a) Truth table



(b) K-map (c) NAND logic

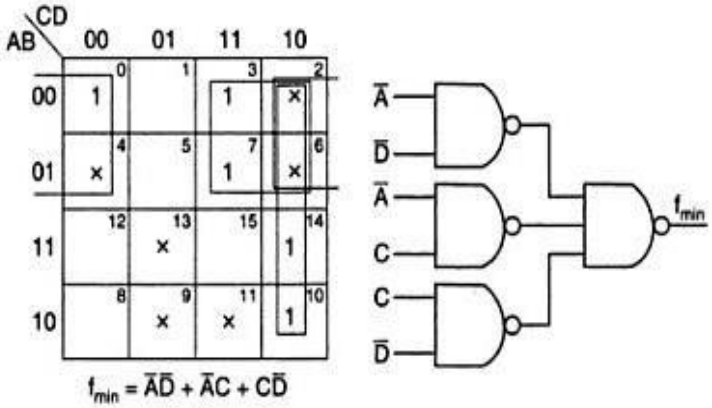
Truth table, K-map and logic diagram for the SOP circuit.

Example: Design of a SOP circuit to detect the decimal numbers 0, 2, 4, 6, 8 in a 4-bit 5211 BCD code input:

Solution:

Decimal number	5211 code A	B	C	D	Output f
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	1	1
3	0	1	0	1	0
4	0	1	1	1	1
5	1	0	0	0	0
6	1	0	1	0	1
7	1	1	0	0	0
8	1	1	1	0	1
9	1	1	1	1	0

(a) Truth table



(b) K-map (c) Logic diagram

Truth table, K-map and logic diagram for the SOP circuit.

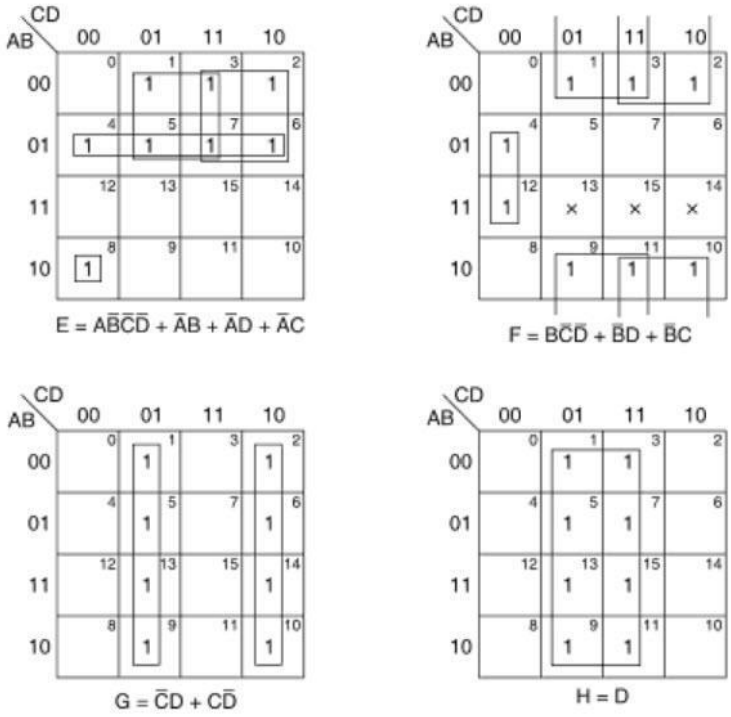
Example: Design of a Combinational circuit to produce the 2's complement of a 4-bit binary number.

Solution:

Input A	B	C	D	Output E	F	G	H
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

(a) Conversion table

$E = \Sigma\, m(1, 2, 3, 4, 5, 6, 7, 8)$, $F = \Sigma\, m(1, 2, 3, 4, 9, 10, 11, 12)$,
$G = \Sigma\, m(1, 2, 5, 6, 9, 10, 13, 14)$, $H = \Sigma\, m(1, 3, 5, 7, 9, 11, 13, 15)$



(b) K-maps

After simplification the minimal expressions are

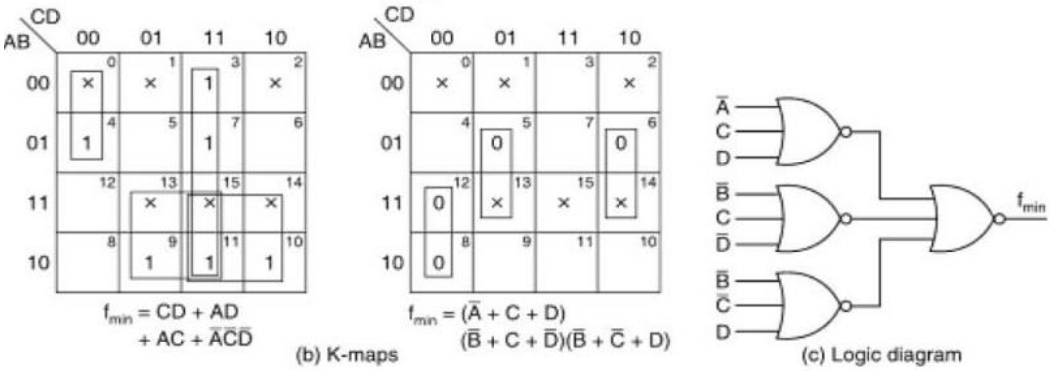
$E = A\bar{B}\bar{C}\bar{D} + \bar{A}B + \bar{A}D + \bar{A}C$, $F = B\bar{C}\bar{D} + \bar{B}D + \bar{B}C$, $G = \bar{C}D + C\bar{D}$, $H = D$

Example: Design of a circuit to detect the decimal numbers 0, 1, 4, 6, 7 and 8 in a 4-bit XS-3 code input.

Solution:

Decimal number	4-bit excess-3 A	B	C	D	Output f
0	0	0	1	1	1
1	0	1	0	0	1
2	0	1	0	1	0
3	0	1	1	0	0
4	0	1	1	1	1
5	1	0	0	0	0
6	1	0	0	1	1
7	1	0	1	0	1
8	1	0	1	1	1
9	1	1	0	0	0

(a) Truth table

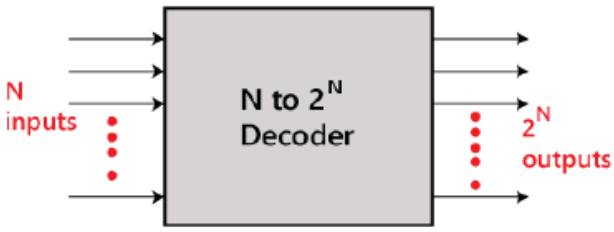


(b) K-maps (c) Logic diagram

Figure Truth table, K-maps and logic diagram for the circuit.

DECODERS

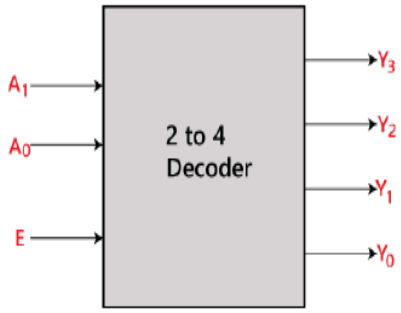
Discrete quantities of information are represented in digital systems by binary codes. A binary code of n bits is capable of representing up to 2^n distinct elements of coded information. **A decoder is a combinational circuit that converts binary information from n input lines to a maximum of 2^n unique output lines.** If the n -bit coded information has unused combinations, the decoder may have fewer than 2^n outputs.



The decoders presented here are called **n -to- m -line decoders, where $m \leq 2^n$**. Their purpose is to generate the 2^n (or fewer) minterms of n input variables. Each combination of inputs will assert a unique output. The name decoder is also used in conjunction with other code converters, such as a BCD-to-seven-segment decoder.

2 to 4-line decoder:

In the 2 to 4-line decoder, there is a total of three inputs, i.e., A0, and A1 and E and four outputs, i.e., Y0, Y1, Y2, and Y3. For each combination of inputs, when the enable 'E' is set to 1, one of these four outputs will be 1. The block diagram and the truth table of the 2 to 4-line decoder are given below.



(a) Block Diagram

Enable	INPUTS		OUTPUTS			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

(b) Truth table

The logical expression of the term Y0, Y0, Y2, and Y3 is as follows:

Y3=E.A1.A0
Y2=E.A1.A0'
Y1=E.A1'.A0
Y0=E.A1'.A0'

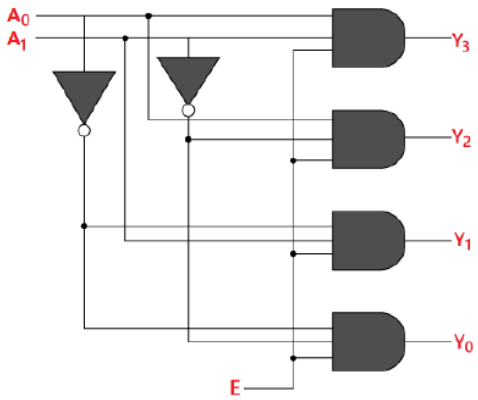
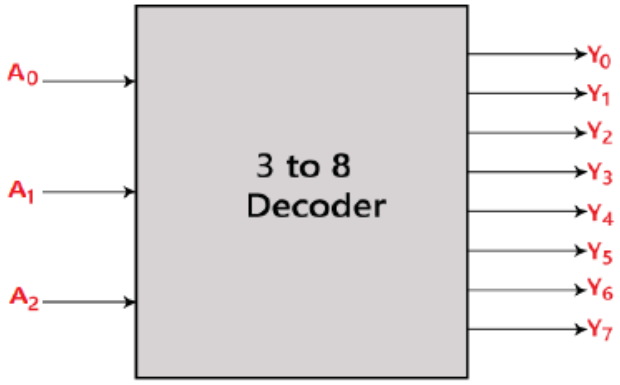


Figure: Logical circuit of 2 to 4-line decoder

3 to 8-line decoder:

The 3 to 8-line decoder is also known as **binary to octal decoder**. The input variables represent a binary number, and the outputs represent the eight digits of a number in the octal number system. In a 3 to 8-line decoder, there is a total of eight outputs, i.e., Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7 and three outputs, i.e., A0, A1, and A2. This circuit has an enable input 'E'. Just like 2 to 4-line decoder, when enable 'E' is set to 1, one of these eight outputs will be 1. The block diagram and the truth table of the 3 to 8 line encoder are given below.



(a) Block Diagram

Enable	INPUTS			Outputs							
E	A_2	A_1	A_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

(b) Truth table

The logical expression of the term Y0, Y1, Y2, Y3, Y4, Y5, Y6, and Y7 is as follows:

Y0=A0'.A1'.A2'
Y1=A0.A1'.A2'
Y2=A0'.A1.A2'
Y3=A0.A1.A2'
Y4=A0'.A1'.A2
Y5=A0.A1'.A2
Y6=A0'.A1.A2
Y7=A0.A1.A2

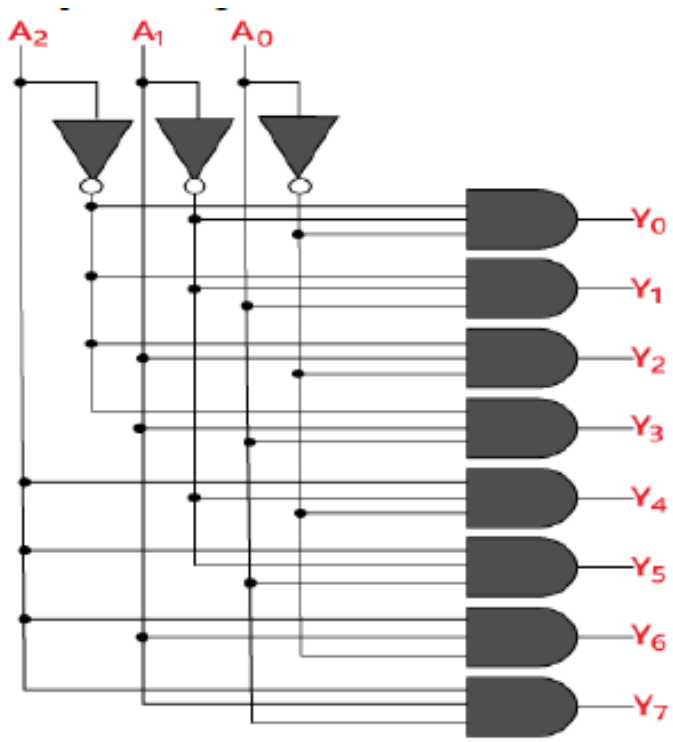


Figure: Logic circuit of 3 to 8 line decoder

4 to 16 line Decoder:

In the 4 to 16 line decoder, there is a total of 16 outputs, i.e., Y0, Y1, Y2 …Y15 and four inputs, i.e., A0, A1, A2, and A3. The 4 to 16 line decoder can be constructed using either 2 to 4 decoder or 3 to 8 decoder. There is the following formula used to find the required number of lower-order decoders.

Required number of lower order decoders = m2/m1

m1 = 8

m2 = 16

Required number of 3 to 8 decoders = 16/8 = 2

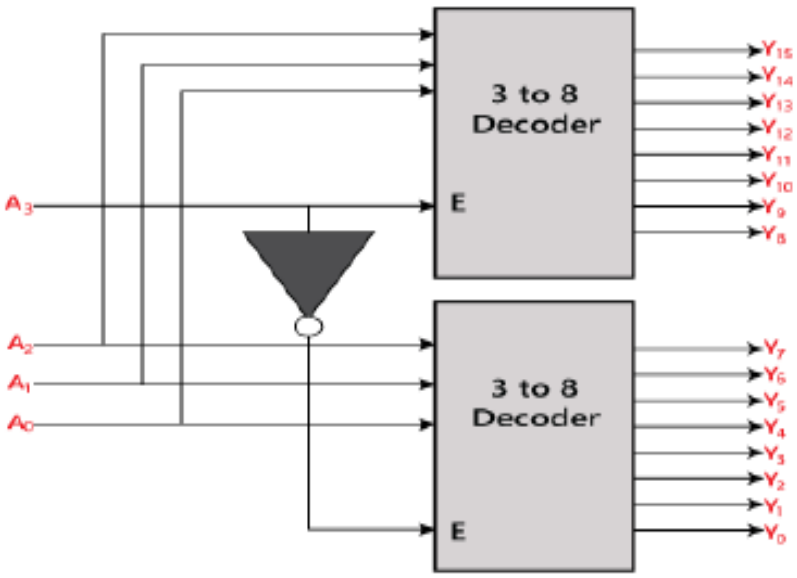


Figure: Implementation of 4 to 16 line decoder using two 3 to 8 line decoders.

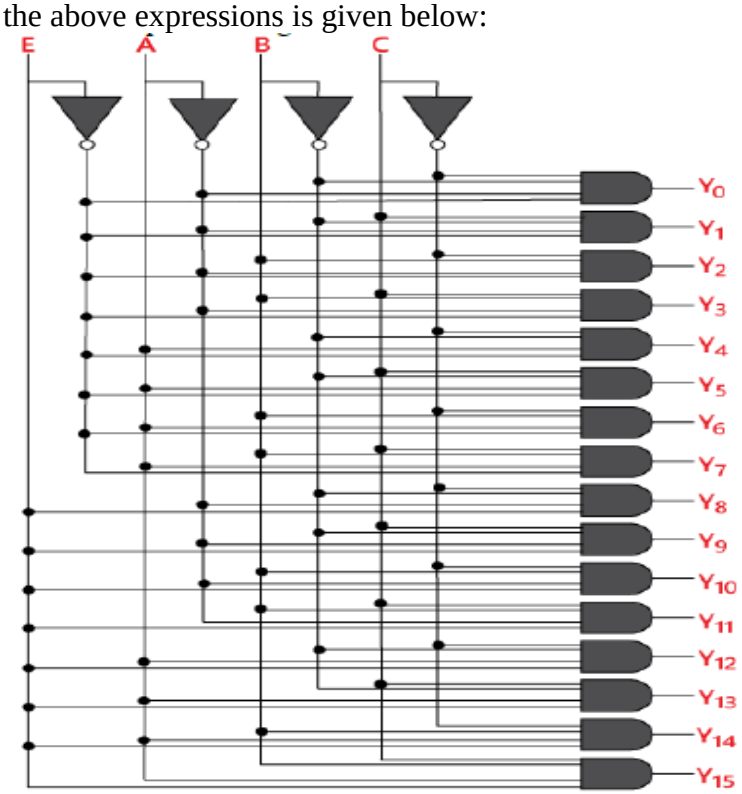
INPUTS				OUTPUTS															
A_3	A_2	A_1	A_0	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure: Truth table for 4 to 16 line decoder

The logical expression of the term A0, A1, A2… A15 are as follows:

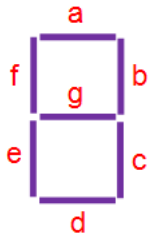
Y0=A0'.A1'.A2'.A3'
Y1=A0'.A1'.A2'.A3
Y2=A0'.A1'.A2.A3'
Y3=A0'.A1'.A2.A3
Y4=A0'.A1.A2'.A3'
Y5=A0'.A1.A2'.A3
Y6=A0'.A1.A2.A3'
Y7=A0'.A1.A2.A3
Y8=A0.A1'.A2'.A3'
Y9=A0.A1'.A2'.A3
Y10=A0.A1'.A2.A3'
Y11=A0.A1'.A2.A3
Y12=A0.A1.A2'.A3'
Y13=A0.A1.A2'.A3
Y14=A0.A1.A2.A3'
Y15=A0.A1.A2'.A3

Logical circuit of the above expressions is given below:



BCD TO SEVEN SEGMENT DECODER:

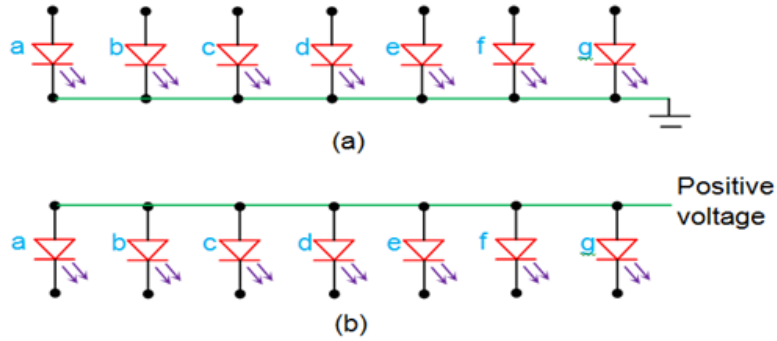
BCD (Binary Coded Decimal) is an encoding scheme which represents each of the decimal numbers by its equivalent 4-bit binary pattern. Seven segment displays comprise of seven individual segments formed by either Light Emitting Diodes (LEDs) or Liquid Crystal Displays (LCDs) arranged in a definite pattern as shown below:



For the display to work, these segments are to be driven by the certain logic level at their input. Depending on this, seven segment displays are found to be of two types, i.e. **common cathode type** and **common anode type.**

A common cathode display (Figure a) has all the cathode terminals of its LED segments tied together. Further, this is grounded and hence is considered to be at logic 0 state. This means that in order to light up an LED, one needs to drive it high.

On the other hand, a common anode display (Figure b) has all its anode terminals connected together which is further driven high by connecting it to a positive supply voltage. Hence for this kind of display to work, one has to drive low on the cathode terminals of the individual LED segments.



(a) Common cathode type display (b) Common anode type display

BCD to seven segment decoder is a circuit used to convert the input BCD into a form suitable for the display. It has four input lines (A, B, C and D) and 7 output lines (a, b, c, d, e, f and g) as shown in below figure.

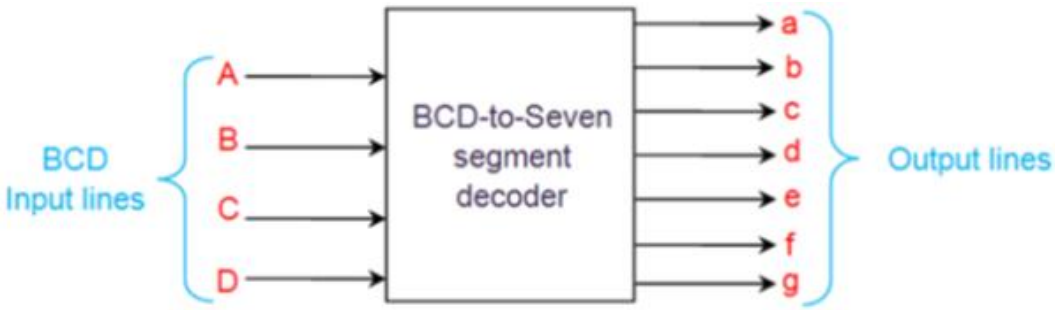


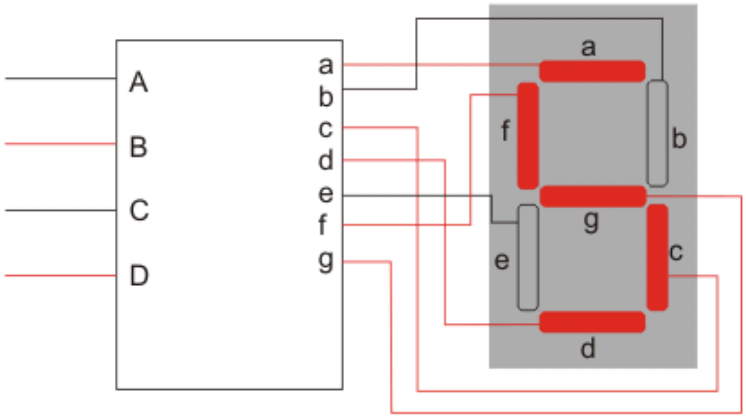
Figure: Block diagram of BCD to Seven segment decoder

Considering common cathode type of arrangement, the truth table for the decoder can be given as in below table.

Table: Truth table for common cathode type BCD to seven segment decoder

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	1	1	1	1	1	1	0	0	1	3
4	0	1	0	0	0	1	1	0	0	1	1	4
5	0	1	0	1	1	0	1	1	0	1	1	5
6	0	1	1	0	1	0	1	1	1	1	1	6
7	0	1	1	1	1	1	1	0	0	0	0	7
8	1	0	0	0	1	1	1	1	1	1	1	8
9	1	0	0	1	1	1	1	1	0	1	1	9

The above table indicates the segments which are to be driven high to obtain certain decimal digit at the output of the seven segment display. However, it is to be noted that in the case of common anode type, the only change will be to interchange ones and zeros on the table. This means that from the truth table so obtained one can get to know where low has to be driven so as to obtain the required digit at the output.



Example: Implement full adder circuit with a decoder and two OR gates.
Solution: From the truth table of the full adder, we obtain the functions for the combinational circuit in sum-of-minterms form:

$$S(x, y, z) = \Sigma (1, 2, 4, 7)$$
$$C(x, y, z) = \Sigma (3, 5, 6, 7)$$

Since there are three inputs and a total of eight minterms, we need a three-to-eight-line decoder. The implementation is shown below. The decoder generates the eight minterms for x, y, and z. The OR gate for output S forms the logical sum of minterms 1, 2, 4, and 7. The OR gate for output C forms the logical sum of minterms 3, 5, 6, and 7.

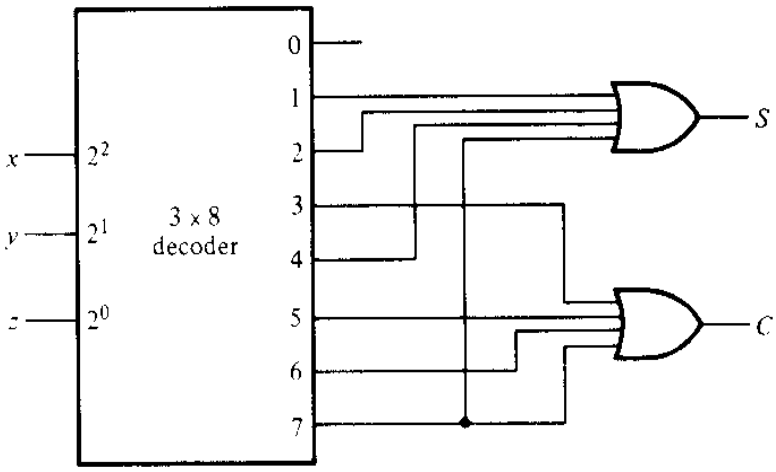


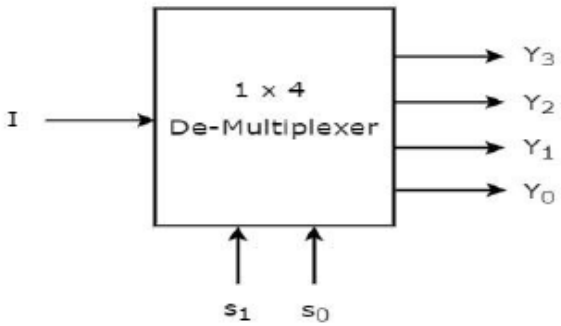
Figure: Implementation of full adder with a decoder

DE-MULTIPLEXERS.

De-Multiplexer is a combinational circuit that performs the reverse operation of Multiplexer. It has single input, 'n' selection lines and maximum of 2^n outputs. The input will be connected to one of these outputs based on the values of selection lines. Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination can select only one output. De-Multiplexer is also called as De-Mux.

1x4 De-Multiplexer:

1x4 De-Multiplexer has one input I, two selection lines, S1 & S0 and four outputs Y3, Y2, Y1 &Y0. The block diagram of 1x4 De-Multiplexer is shown in the following figure.



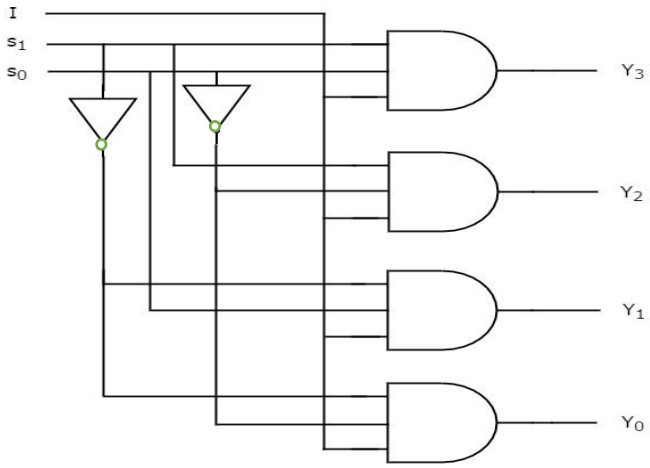
The single input 'I' will be connected to one of the four outputs, Y3 to Y0 based on the values of selection lines S1 & S0. The truth table of 1x4 De-Multiplexer is shown below.

Selection Inputs		Outputs			
S1	S0	Y3	Y2	Y1	Y0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

From the above Truth table, we can directly write the Boolean functions for each output as

Y3=S1.S0.I
Y2=S1.S0′.I
Y1=S1′.S0.I
Y0=S1′.S0′ I

We can implement these Boolean functions using Inverters & 3-input AND gates. The circuit diagram of 1x4 De-Multiplexer is shown in the following figure.



We can easily understand the operation of the above circuit. Similarly, you can implement 1x8 De-Multiplexer and 1x16 De-Multiplexer by following the same procedure.

Implementation of Higher-order De-Multiplexers:

Now, let us implement the following two higher-order De-Multiplexers using lower-order De-Multiplexers.

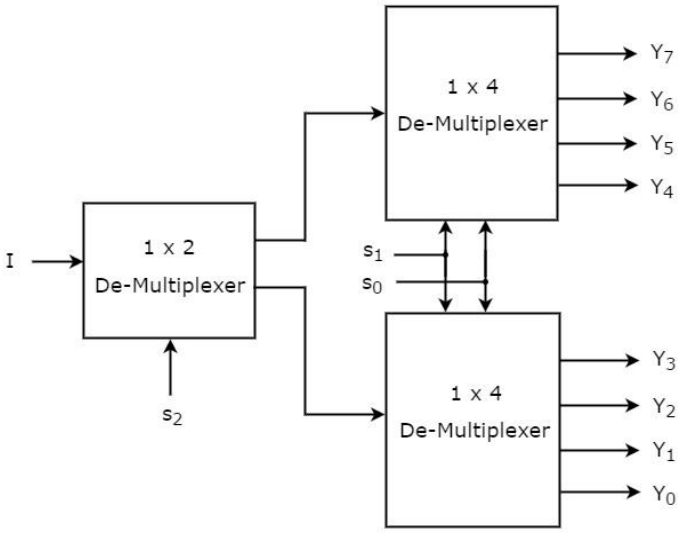
1x8 De-Multiplexer

In this section, let us implement 1x8 De-Multiplexer using 1x4 De-Multiplexers and 1x2 De-Multiplexer. We know that 1x4 De-Multiplexer has single input, two selection lines and four outputs. Whereas, 1x8 De-Multiplexer has single input, three selection lines and eight outputs. So, we require two 1x4 De-Multiplexers in second stage in order to get the final eight outputs. Since, the number of inputs in second stage is two, we require 1x2 De-Multiplexer in first stage so that the outputs of first stage will be the inputs of second stage. Input of this 1x2 De-Multiplexer will be the overall input of 1x8 De-Multiplexer.

Let the 1x8 De-Multiplexer has one input I, three selection lines S2, S1 & S0 and outputs Y7 to Y0. The Truth table of 1x8 De-Multiplexer is shown below.

Selection Inputs			Outputs							
s2	s1	s0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

We can implement 1x8 De-Multiplexer using lower order Multiplexers easily by considering the above Truth table. The block diagram of 1x8 De-Multiplexer is shown in the following figure.



The common selection lines, S1 & S0 are applied to both 1x4 De-Multiplexers. The outputs of upper 1x4 De-Multiplexer are Y7 to Y4 and the outputs of lower 1x4 De-Multiplexer are Y3 to Y0. The other selection line, S2 is applied to 1x2 De-Multiplexer.

If S2 is zero, then one of the four outputs of lower 1x4 De-Multiplexer will be equal to input, I based on the values of selection lines S1 & S0. Similarly, if S2 is one, then one of the four outputs of upper 1x4 De-Multiplexer will be equal to input, I based on the values of selection lines S1 & S0.

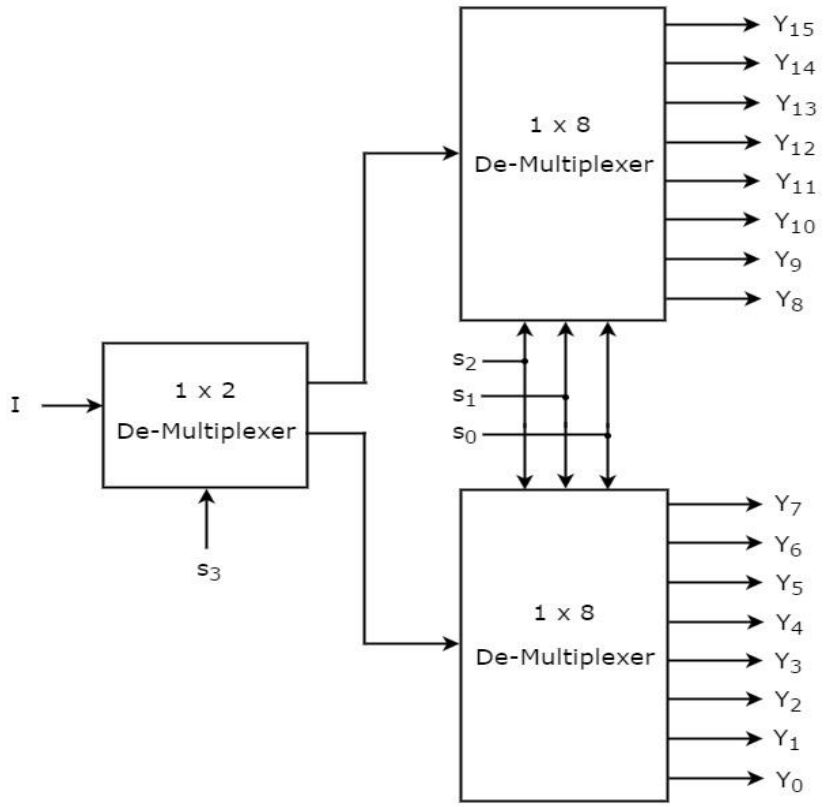
1x16 De-Multiplexer

In this section, let us implement 1x16 De-Multiplexer using 1x8 De-Multiplexers and 1x2 De-Multiplexer. We know that 1x8 De-Multiplexer has single input, three selection lines and eight outputs. Whereas, 1x16 De-Multiplexer has single input, four selection lines and sixteen outputs.

So, we require two 1x8 De-Multiplexers in second stage in order to get the final sixteen outputs. Since, the number of inputs in second stage is two, we require 1x2 De-Multiplexer in first stage so that the outputs of first stage will be the inputs of second stage. Input of this 1x2 De-Multiplexer will be the overall input of 1x16 De-Multiplexer.

Let the 1x16 De-Multiplexer has one input I, four selection lines S3, S2, S1 & S0 and outputs Y15 to Y0. The block diagram of 1x16 De-Multiplexer using lower order Multiplexers is shown in the following figure.

The common selection lines S2, S1 & S0 are applied to both 1x8 De-Multiplexers. The outputs of upper 1x8 De-Multiplexer are Y15 to Y8 and the outputs of lower 1x8 De-Multiplexer are Y7 to Y0. The other selection line, S3 is applied to 1x2 De-Multiplexer. If S3 is zero, then one of the eight outputs of lower 1x8 De-Multiplexer will be equal to input, I based on the values of selection lines S2, S1 & S0. Similarly, if S3 is one, then one of the 8 outputs of upper 1x8 De-Multiplexer will be equal to input, I based on the values of selection lines S2, S1 & S0.



A decoder with enable input can function as a demultiplexer— a circuit that receives information from a single line and directs it to one of 2^n possible output lines. The selection of a specific output is controlled by the bit combination of n selection lines. The below decoder can function as a one-to-four-line demultiplexer when E is taken as a data input line, A and B are taken as the selection inputs. The single input variable E has a path to all four outputs, but the input information is directed to only one of the output lines, as specified by the binary combination of the two selection lines A and B. This feature can be verified from the truth table of the circuit.

For example, if the selection lines AB = 10, output D2 will be the same as the input value E, while all other outputs are maintained at 1. Because decoder and demultiplexer operations are obtained from the same circuit, a decoder with an enable input is referred to as a **decoder – demultiplexer**. It is the enable input that makes the circuit a demultiplexer; the decoder itself can use AND, NAND, or NOR gates.

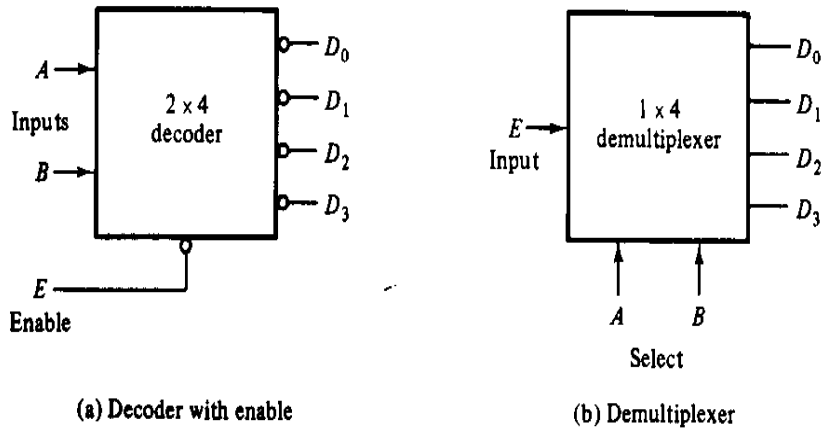


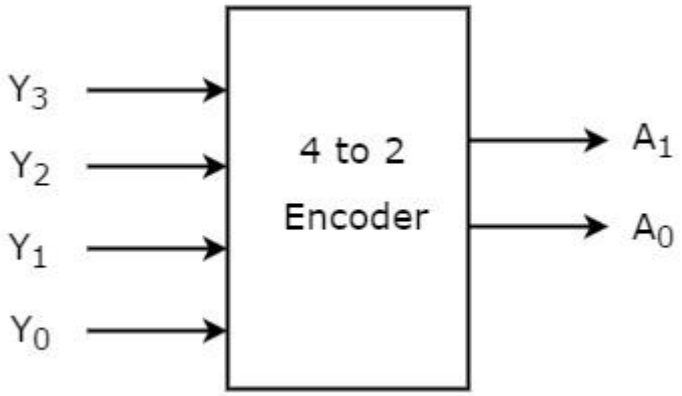
Figure: Block diagram of decoder with enable and Demultiplexer

ENCODERS

An Encoder is a combinational circuit that performs the reverse operation of decoder. It has maximum of 2^n input lines and 'n' output lines. It will produce a binary code equivalent to the input, which is active high. Therefore, the encoder encodes 2^n input lines with 'n' bits. It is optional to represent the enable signal in encoders.

4 TO 2 ENCODER:

Let 4 to 2 Encoder has four inputs Y3, Y2, Y1 & Y0 and two outputs A1 & A0. The block diagram of 4 to 2 encoder is shown below.



At any time, only one of these 4 inputs can be '1' in order to get the respective binary code at the output. The truth table of 4 to 2 encoder is shown below.

Inputs				Outputs	
Y0	Y1	Y2	Y3	A0	A1
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

From Truth table, we can write the Boolean functions for each output as

A0= Y3+Y2

A1=Y3+Y1

We can implement the above two Boolean functions by using two input OR gates. The circuit diagram of 4 to 2 encoder is shown in the below figure. These OR gates encode the four inputs with two bits.

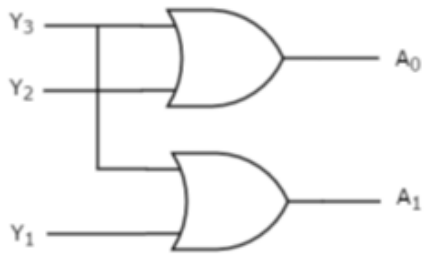


Figure: Logic diagram of 4 to 2 encoder

OCTAL TO BINARY ENCODER:

Octal to binary Encoder has eight inputs, Y7 to Y0 and three outputs A2, A1 & A0. Octal to binary encoder is nothing but 8 to 3 encoder. The block diagram of octal to binary encoder is shown in the below figure.

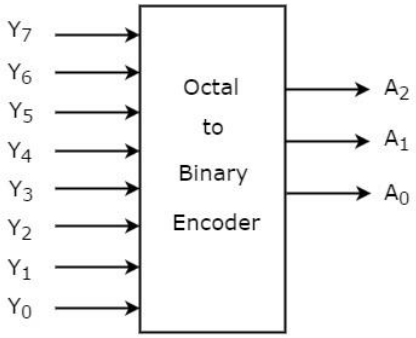


Figure: Block diagram of octal to binary (8 to 3) encoder

At any time, only one of these eight inputs can be '1' in order to get the respective binary code. The truth table of octal to binary encoder is shown below.

Inputs								Outputs		
Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7	A0	A1	A2
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

From Truth table, we can write the Boolean functions for each output as

A0=Y7+Y6+Y5+Y4

A1=Y7+Y6+Y3+Y2

A2=Y7+Y5+Y3+Y1

We can implement the above Boolean functions by using four input OR gates. The circuit diagram of octal to binary encoder is shown in the following figure.

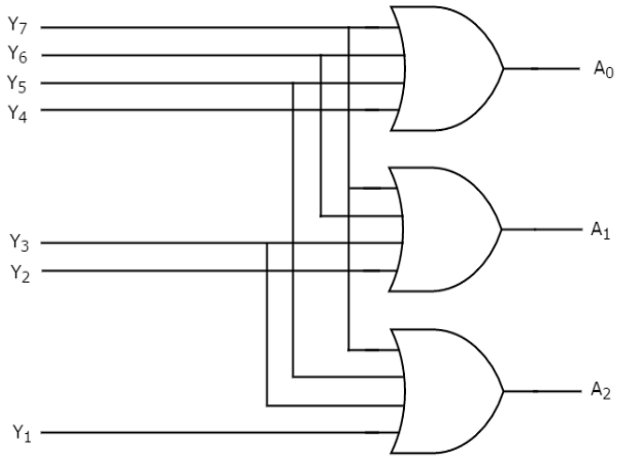
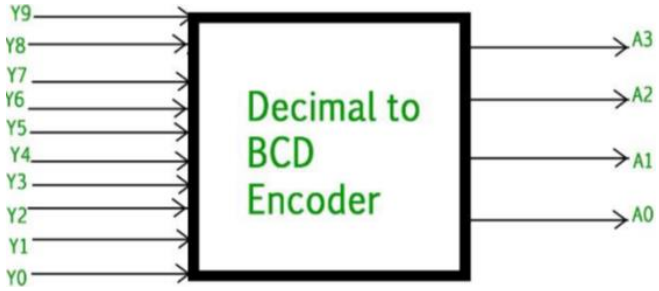


Figure: Logic diagram of octal to binary encoder

DECIMAL TO BCD ENCODER:

The decimal to binary encoder usually consists of **10 input lines** and 4 output lines. Each input line corresponds to the each decimal digit and 4 outputs correspond to the BCD code. This encoder accepts the decoded decimal data as an input and encodes it to the BCD output which is available on the output lines. The figure below shows the logic symbol of decimal to BCD encoder.



The truth table for decimal to BCD encoder is as follows:

INPUTS										OUTPUTS			
Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7	Y8	Y9	A0	A1	A2	A3
1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	1	1	1
0	0	0	0	0	0	0	0	1	0	1	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	1

Logical expression for A3, A2, A1 and A0:

A0 = Y9 + Y8

A1 = Y7 + Y6 + Y5 +Y4

A2 = Y7 + Y6 + Y3 +Y2

A3 = Y9 + Y7 +Y5 +Y3 + Y1

The above Boolean functions can be implemented using OR gates:

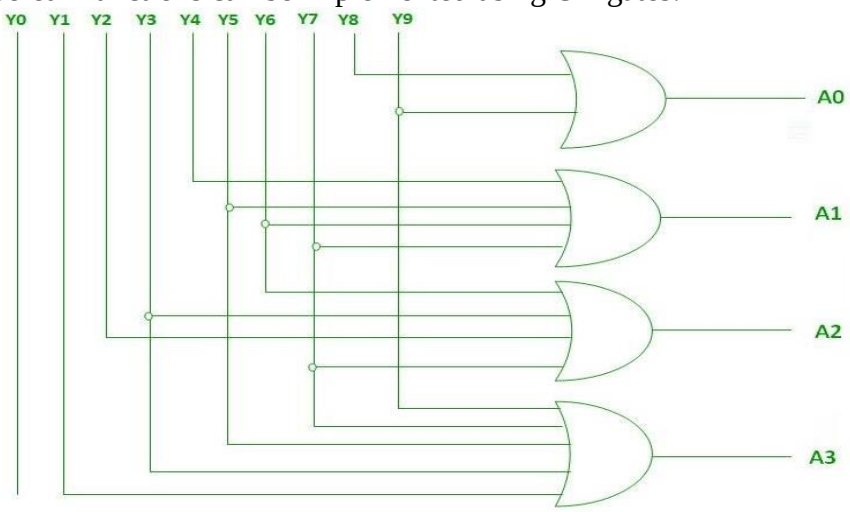


Figure: Block diagram of decimal to BCD encoder

Drawbacks of Encoder:

The encoder has the limitation that only one input can be active at any given time. If two inputs are active simultaneously, the output produces an undefined combination. **For example, if Y3 and Y6 are 1 simultaneously, the output of the encoder will be 111 because all three outputs are equal to 1. The output 111 does not represent either binary 3 or binary 6.** To resolve this ambiguity, encoder circuits must establish an input priority to ensure that only one input is encoded. If we establish a higher priority for inputs with higher subscript numbers, and if both Y3 and Y6 are 1 at the same time, the output will be 110 because Y6 has higher priority than Y3.

Another ambiguity in the octal-to-binary encoder is that an output with all 0's is generated when all the inputs are 0; but this output is the same as when Y0 is equal to 1. The discrepancy can be resolved by providing one more output to indicate whether at least one input is equal to 1.

So, to overcome these difficulties, we should assign priorities to each input of encoder. Then, the output of encoder will be the binary code corresponding to the active High inputs, which has higher priority. This encoder is called as priority encoder.

PRIORITY ENCODER

A **priority encoder** is an encoder circuit that includes the priority function. The operation of the priority encoder is such that **if two or more inputs are equal to 1 at the same time, the input having the highest priority will take precedence**. The truth table of a four-input priority encoder is given in below table. In addition to the two outputs x and y, the circuit has a **third output** designated by **V**; this is a valid bit indicator that **is set to 1 when one or more inputs are equal to 1**.

If all inputs are 0, there is no valid input and V is equal to 0. The other two outputs are not inspected when V equals 0 and are specified as don't-care conditions. Note that whereas X's in output columns represent don't-care conditions, the X's in the input columns are useful for representing a truth table in condensed form. Instead of listing all 16 minterms of four variables, the truth table uses an X to represent either 1 or 0. For example, X 100 represents the two minterms 0100 and 1100.

Table: Truth table of a priority encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

According to above truth table, the higher the subscript number, the higher the priority of the input. Input **D_3 has the highest priority**, so, regardless of the values of the other inputs, when this input is 1, the output for xy is 11 (binary 3). D_2 has the next priority level. The output is 10 if D_2 = 1, provided that D_3 = 0, regardless of the values of the other two lower priority

inputs. The output for D_1 is generated only if higher priority inputs are 0, and so on down the priority levels.

The maps for simplifying outputs x and y are shown below. The minterms for the two functions are derived from above truth table. Although the table has only five rows, when each X in a row is replaced first by 0 and then by 1, we obtain all 16 possible input combinations. For example, the fourth row in the table, with inputs XX10, represents the four minterms 0010, 0110, 1010, and 1110.

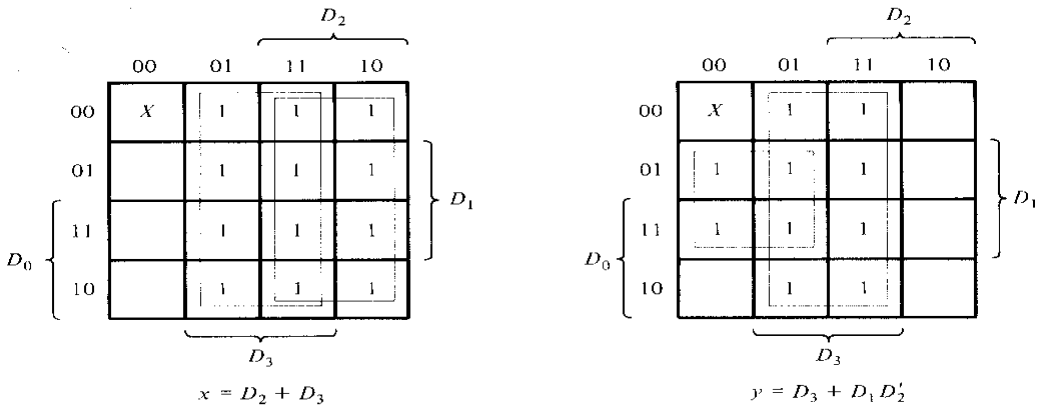


Figure: Maps for a priority encoder

The simplified Boolean expressions for the priority encoder are obtained from the maps. The condition for output V is an OR function of all the input variables. The priority encoder is implemented below according to the following Boolean functions:

$$\mathbf{x = D_2 + D_3}$$
$$\mathbf{y = D_3 + D_1 D'_2}$$
$$\mathbf{V = D_0 + D_1 + D_2 + D_3}$$

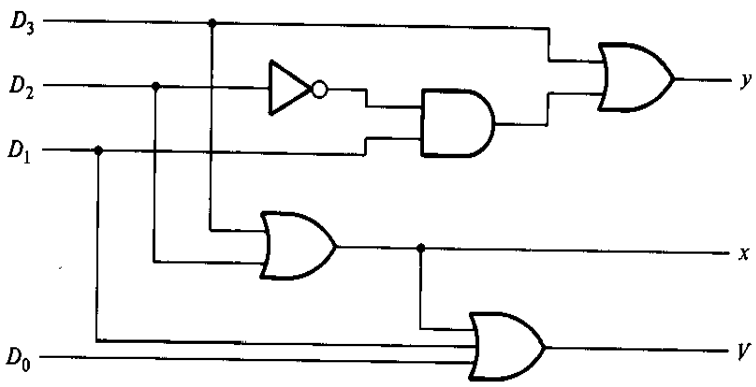


Figure: 4 input priority encoder

MULTIPLEXERS

Multiplexer is a combinational circuit that has **maximum of 2^n data inputs, 'n' selection lines and single output line**. One of these data inputs will be connected to the output based on the values of selection lines. Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination will select only one data input. Multiplexer in short called as MUX.

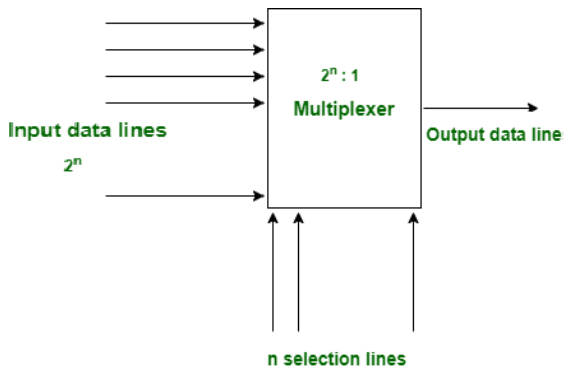


Figure: Block diagram of Multiplexer

Two – to - one line multiplexer (2 × 1 Multiplexer)

A two-to-one-line multiplexer connects one of two 1-bit sources to a common destination, as shown below. The circuit has two data input lines, one output line, and one selection line S. When S = 0, the upper AND gate is enabled and I_0 has a path to the output. When S = 1, the lower AND gate is enabled and I_1 has a path to the output. The multiplexer acts like an electronic switch that selects one of two sources. The block diagram of a multiplexer is sometimes depicted by a wedge-shaped symbol, as shown in Fig. (b). It suggests visually how a selected one of multiple data sources is directed into a single destination. The multiplexer is often labeled "MUX" in block diagrams.

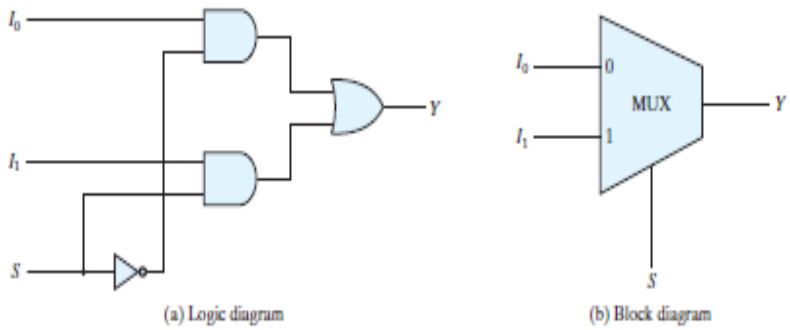


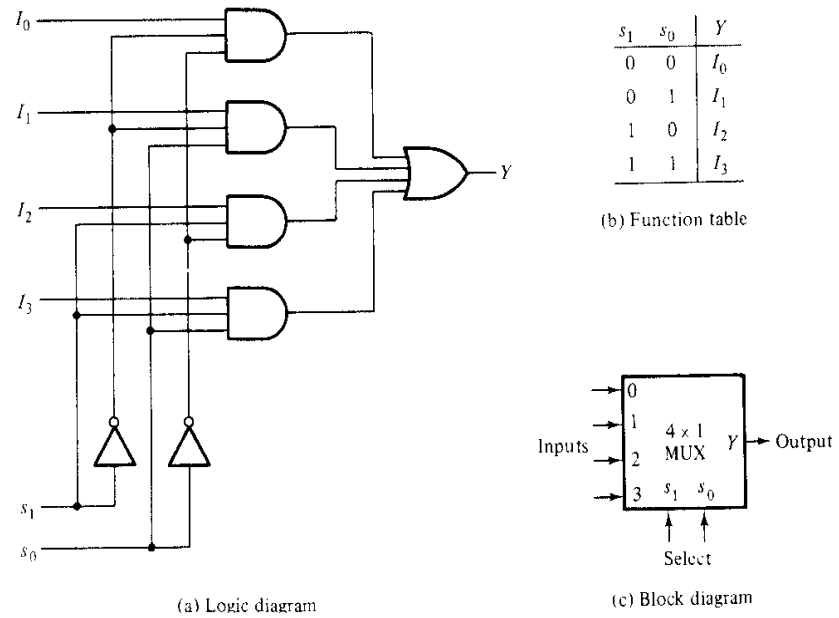
Figure: Two – to - one line multiplexer

Four – to - one line multiplexer (4 × 1 Multiplexer)

A four-to-one-line multiplexer is shown below. Each of the four inputs, I_0 through I_3, is applied to one input of an AND gate. Selection lines S_1 and S_0 are decoded to select a particular AND gate. The outputs of the AND gates are applied to a single OR gate that provides the one-line output. The function table lists the input that is passed to the output for each combination of the binary selection values. To demonstrate the operation of the circuit, consider the case when $S_1S_0 = 10$. The AND gate associated with input I_2 has two of its inputs equal to 1 and the third input connected to I_2. The other three AND gates have at least one input equal to 0, which makes their outputs equal to 0. The output of the OR gate is now equal to the value of I_2, providing a path from the selected input to the output. **A multiplexer is also called a data selector**, since it selects one of many inputs and steers the binary information to the output line.

The AND gates and inverters in the multiplexer resemble a decoder circuit, and indeed, they decode the selection input lines. In general, **a 2^n -to-1-line multiplexer is constructed from an n -to-2^n decoder by adding 2^n input lines to it, one to each AND gate**. The outputs of the AND gates are applied to a single OR gate. The size of a multiplexer is specified by the

number 2^n of its data input lines and the single output line. The n selection lines are implied from the 2^n data lines.



s_1	s_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Figure: Four – to - one line multiplexer

From Truth table, we can directly write the Boolean function for output, Y as

Y = S1′S0′I0+S1′S0I1+S1S0′I2+S1S0I3Y=S1′S0′I0+S1′S0I1+S1S0′I2+S1S0I3

As in decoders**, multiplexers may have an enable input to control the operation of the unit**. When the **enable input is in the inactive state**, the **outputs are disabled**, and when it is in the **active state**, the circuit functions as a **normal multiplexer**.

Eight – to - one line multiplexer (8 × 1 Multiplexer):

An 8-to-1 multiplexer consists of eight data inputs D0 through D7, three input select lines S2 through S0 and a single output line Y. Depending on the select lines combinations, multiplexer decodes the inputs.

The below figure shows the block diagram of an 8-to-1 multiplexer with enable input that enable or disable the multiplexer. Since the number data bits given to the MUX are eight then 3 bits (2^3=8) are needed to select one of the eight data bits.

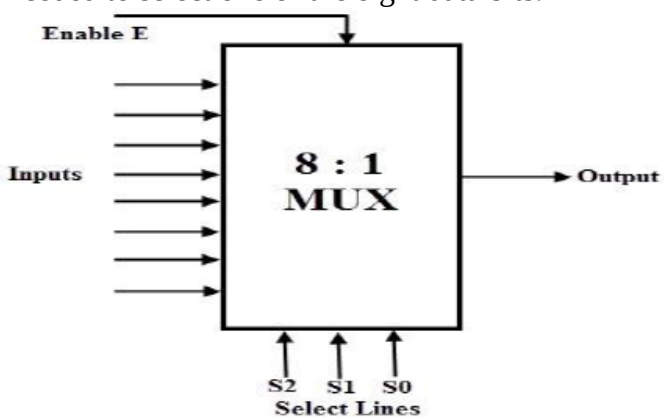
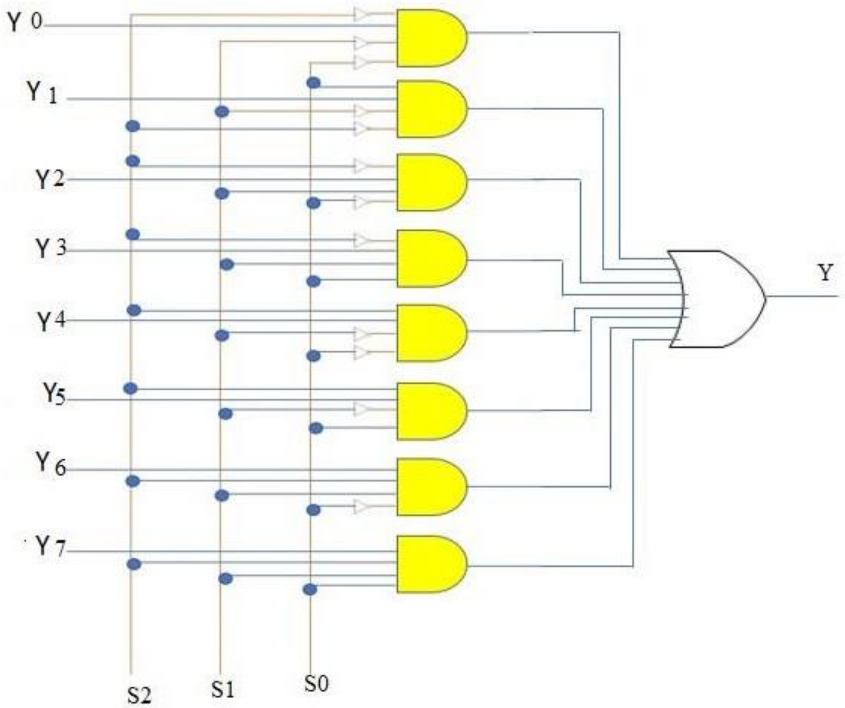


Figure: Block diagram of 8×1 MUX

The truth table for an 8-to1 multiplexer is given below with eight combinations of inputs so as to generate each output corresponds to input. For example, if S2= 0, S1=1 and S0=0 then the data output Y is equal to D2. Similarly the data outputs D0 to D7 will be selected through the combinations of S2, S1 and S0 as shown in below figure.

Selection Inputs			Output
S2	S1	S0	Y
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

From the above truth table, the Boolean equation for the output is given as



Implementation of Higher-order Multiplexers.

Now, let us implement the following two higher-order Multiplexers using lower-order Multiplexers.

8x1 Multiplexer

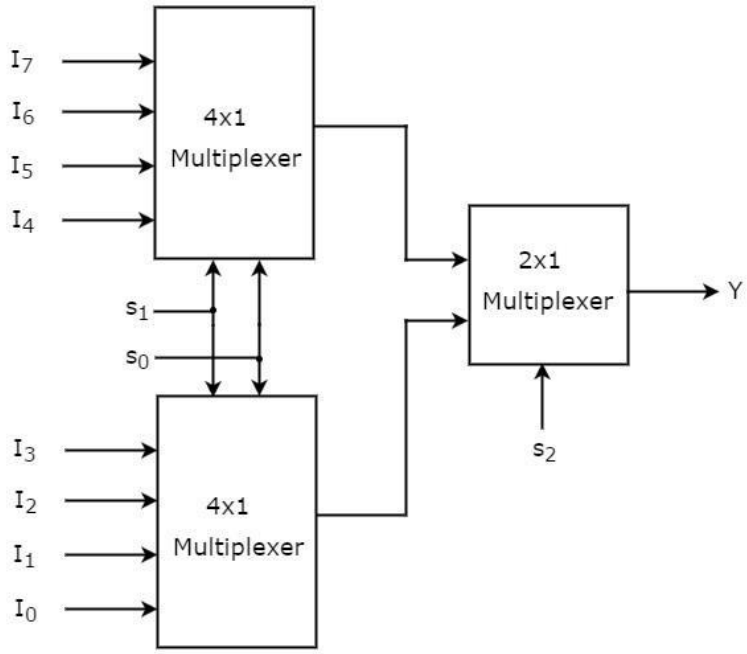
In this section, let us implement 8x1 Multiplexer using 4x1 Multiplexers and 2x1 Multiplexer. We know that 4x1 Multiplexer has 4 data inputs, 2 selection lines and one output. Whereas, 8x1 Multiplexer has 8 data inputs, 3 selection lines and one output.

So, we require two 4x1 Multiplexers in first stage in order to get the 8 data inputs. Since, each 4x1 Multiplexer produces one output, we require a 2x1 Multiplexer in second stage by considering the outputs of first stage as inputs and to produce the final output.

Let the 8x1 Multiplexer has eight data inputs I7 to I0, three selection lines S2, S1 & S0 and one output Y. The Truth table of 8x1 Multiplexer is shown below:

Selection Inputs			Output
S2	S1	S0	Y
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3
1	0	0	I4
1	0	1	I5
1	1	0	I6
1	1	1	I7

We can implement 8x1 Multiplexer using lower order Multiplexers easily by considering the above Truth table. The block diagram of 8x1 Multiplexer is shown in the following figure.



The same selection lines, s1 & s0 are applied to both 4x1 Multiplexers. The data inputs of upper 4x1 Multiplexer are I7 to I4 and the data inputs of lower 4x1 Multiplexer are I3 to I0. Therefore, each 4x1 Multiplexer produces an output based on the values of selection lines, S1 & S0.

The outputs of first stage 4x1 Multiplexers are applied as inputs of 2x1 Multiplexer that is present in second stage. The other selection line, S2 is applied to 2x1 Multiplexer. If S2 is zero, then the output of 2x1 Multiplexer will be one of the 4 inputs I3 to I0 based on the values of selection lines S1 & S0. If S2 is one, then the output of 2x1 Multiplexer will be one of the 4 inputs I7 to I4 based on the values of selection lines S1 & S0.

Therefore, the overall combination of two 4x1 Multiplexers and one 2x1 Multiplexer performs as one 8x1 Multiplexer.

16x1 Multiplexer

In this section, let us implement 16x1 Multiplexer using 8x1 Multiplexers and 2x1 Multiplexer. We know that 8x1 Multiplexer has 8 data inputs, 3 selection lines and one output. Whereas, 16x1 Multiplexer has 16 data inputs, 4 selection lines and one output. So, we require two 8x1 Multiplexers in first stage in order to get the 16 data inputs. Since, each 8x1 Multiplexer produces one output, we require a 2x1 Multiplexer in second stage by considering the outputs of first stage as inputs and to produce the final output.

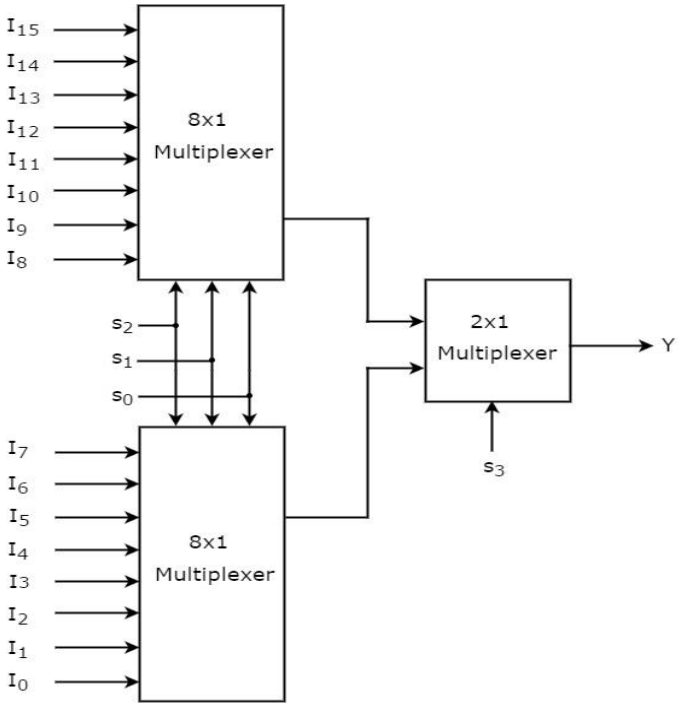
Let the 16x1 Multiplexer has sixteen data inputs I15 to I0, four selection lines S3 to S0 and one output Y. The Truth table of 16x1 Multiplexer is shown below.

Selection Inputs				Output
S3	S2	S1	S0	Y
0	0	0	0	I0
0	0	0	1	I1
0	0	1	0	I2
0	0	1	1	I3
0	1	0	0	I4
0	1	0	1	I5
0	1	1	0	I6
0	1	1	1	I7
1	0	0	0	I8
1	0	0	1	I9
1	0	1	0	I10
1	0	1	1	I11
1	1	0	0	I12
1	1	0	1	I13
1	1	1	0	I14
1	1	1	1	I15

We can implement 16x1 Multiplexer using lower order Multiplexers easily by considering the above Truth table. The block diagram of 16x1 Multiplexer is shown in the following figure. The same selection lines, s2, s1 & s0 are applied to both 8x1 Multiplexers. The data inputs of upper 8x1 Multiplexer are I15 to I8 and the data inputs of lower 8x1 Multiplexer are I7 to I0. Therefore, each 8x1 Multiplexer produces an output based on the values of selection lines, S2, S1 & S0.

The outputs of first stage 8x1 Multiplexers are applied as inputs of 2x1 Multiplexer that is present in second stage. The other selection line, S3 is applied to 2x1 Multiplexer. If S3 is zero, then the output of 2x1 Multiplexer will be one of the 8 inputs Is7 to I0 based on the values of selection lines S2, S1 & S0. If S3 is one, then the output of 2x1 Multiplexer will be one of the 8 inputs I15 to I8 based on the values of selection lines S2, S1 & S0. Therefore, the overall

combination of two 8x1 Multiplexers and one 2x1 Multiplexer performs as one 16x1 Multiplexer.



Multiplexer circuits can be combined with common selection inputs to provide multiple-bit selection logic. As an illustration, a quadruple 2-to-1-line multiplexer is shown below. The circuit has four multiplexers, each capable of selecting one of two input lines. Output Y_0 can be selected to come from either input A_0 or input B_0. Similarly, output Y_1 may have the value of A_1 or B_1, and so on. Input selection line S selects one of the lines in each of the four multiplexers. The enable input E must be active (i.e., asserted) for normal operation.

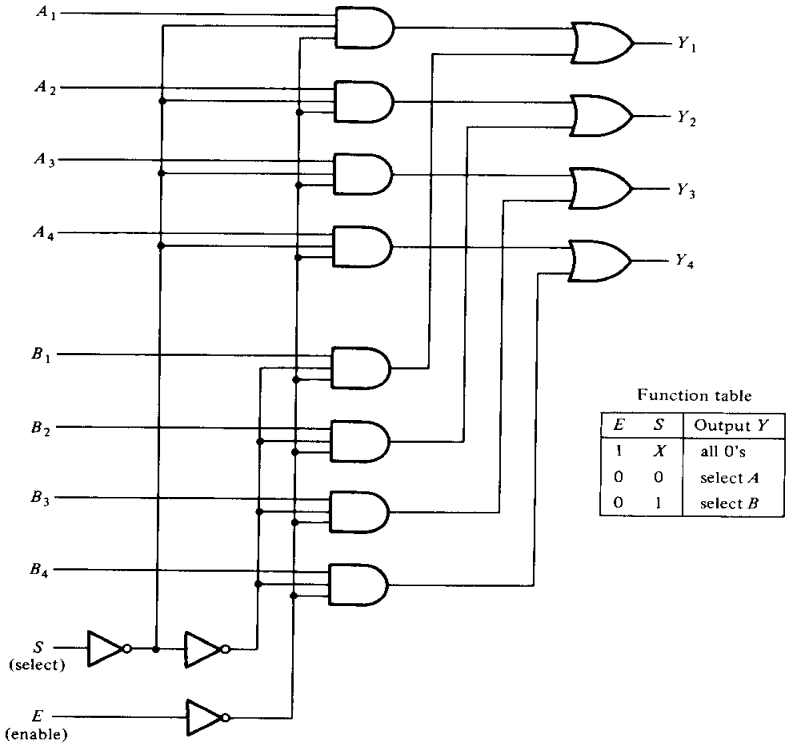


Figure: Quadruple two-to-one-line multiplexer

Although the circuit contains four 2-to-1-line multiplexers, we are more likely to view it as a circuit that selects one of two 4-bit sets of data lines. As shown in the function table, the unit is enabled when E = 0. Then, if S = 0, the four A inputs have a path to the four outputs. If, by contrast, S = 1, the four B inputs are applied to the outputs. The outputs have all 0's when E = 1, regardless of the value of S.

Boolean Function Implementation

An examination of the **logic diagram of a multiplexer reveals that it is essentially a decoder that includes the OR gate within the unit**. **The minterms of a function are generated in a multiplexer by the circuit associated with the selection inputs**. The individual minterms can be selected by the data inputs, thereby providing a method of implementing a Boolean function of n variables with a multiplexer that has n selection inputs and 2^n data inputs, one for each minterm.

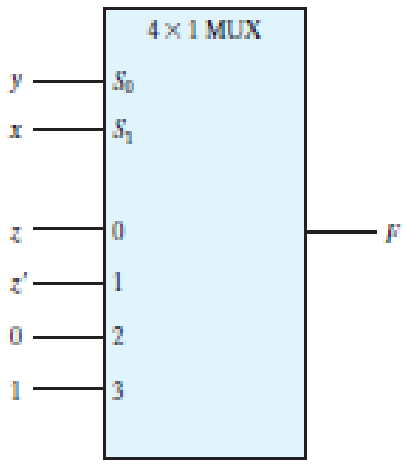
An efficient method for implementing a Boolean function of n variables with a multiplexer that has n - 1 selection inputs is shown here. The first n-1 variables of the function are connected to the selection inputs of the multiplexer. The remaining single variable of the function is used for the data inputs. If the single variable is denoted by z, each data input of the multiplexer will be z, z', 1, or 0.

Example: Implement the Boolean function F (x, y, z) = Σ (1, 2, 6, 7) with a 4 to 1 multiplexer.

Solution: The given function of three variables can be implemented with a four-to-one-line multiplexer as shown below. The two variables **x and y are applied to the selection lines** in that order; x is connected to the S_1 input and y to the S_0 input. The values for the data input lines are determined from the truth table of the function. When xy = 00, output F is equal to z because F = 0 when z = 0 and F = 1 when z = 1. This requires that variable z be applied to data input 0. The operation of the multiplexer is such that when xy = 00, data input 0 has a path to the output, and that makes F equal to z. In a similar fashion, we can determine the required input to data lines 1, 2, and 3 from the value of F when xy = 01, 10, and 11, respectively. This particular example shows all four possibilities that can be obtained for the data inputs.

x	y	z	F	
0	0	0	0	F = z
0	0	1	1	
0	1	0	1	F = z'
0	1	1	0	
1	0	0	0	F = 0
1	0	1	0	
1	1	0	1	F = 1
1	1	1	1	

(a) Truth table

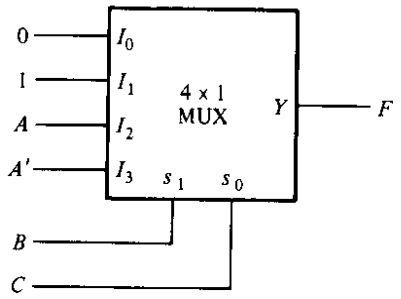


(b) Multiplexer implementation

The general procedure for implementing any Boolean function of n variables with a multiplexer with n -1 selection inputs and 2^n-1 data inputs follows from the above example. To

begin with, Boolean function is listed in a truth table. Then first n - 1 variables in the table are applied to the selection inputs of the multiplexer. For each combination of the selection variables, we evaluate the output as a function of the last variable. This function can be 0, 1, the variable, or the complement of the variable. These values are then applied to the data inputs in the proper order.

Example: Implement the Boolean function F (x, y, z) = Σ (1, 3, 5, 6) with a 4 to 1 multiplexer.
Solution:



(a) Multiplexer implementation

Minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

(b) Truth table

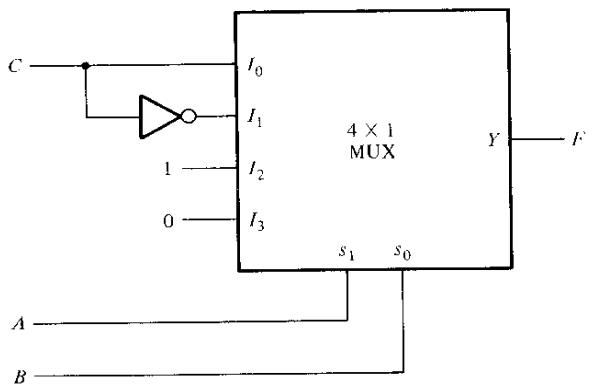
	I_0	I_1	I_2	I_3
A'	0	1	2	3
A	4	5	6	7
	0	1	A	A'

(c) Implementation table

Example: Implement the Boolean function F (x, y, z) = Σ (1, 2, 4, 5) with a 4 to 1 multiplexer.
Solution:

A	B	C	F	
0	0	0	0	F = C
0	0	1	1	
0	1	0	1	F = C'
0	1	1	0	
1	0	0	1	F = 1
1	0	1	1	
1	1	0	0	F = 0
1	1	1	0	

(a) Truth table



(b) Multiplexer implementation

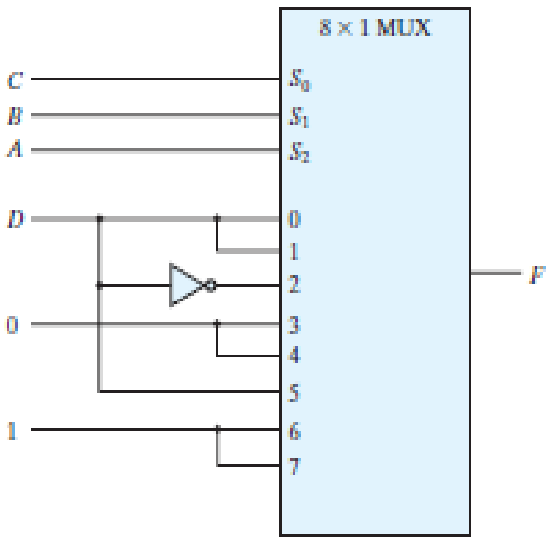
	I_0	I_1	I_2	I_3
C'	0	2	4	6
C	1	3	5	7
	C	C'	1	0

(c) Implementation table

Example: Implement the Boolean function F (A, B, C, D) = Σ (1, 3, 4, 11, 12, 13, 14, 15) with 8 to 1 multiplexer.

Solution:

A	B	C	D	F	
0	0	0	0	0	F = D
0	0	0	1	1	
0	0	1	0	0	F = D
0	0	1	1	1	
0	1	0	0	1	F = D′
0	1	0	1	0	
0	1	1	0	0	F = 0
0	1	1	1	0	
1	0	0	0	0	F = 0
1	0	0	1	0	
1	0	1	0	0	F = D
1	0	1	1	1	
1	1	0	0	1	F = 1
1	1	0	1	1	
1	1	1	0	1	F = 1
1	1	1	1	1	

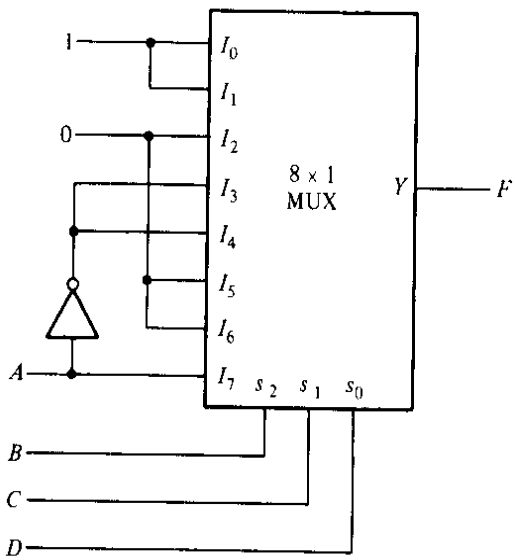


Given function is implemented with a multiplexer with three selection inputs as shown in above. Note that the first variable A must be connected to selection input S_2 so that A, B, and C correspond to selection inputs S_2, S_1, and S_0, respectively. The values for the data inputs are determined from the truth table listed below. The corresponding data line number is determined from the binary combination of ABC. For example, the table shows that when ABC = 101, F = D, so the input variable D is applied to data input 5. The binary constants 0 and 1 correspond to two fixed signal values.

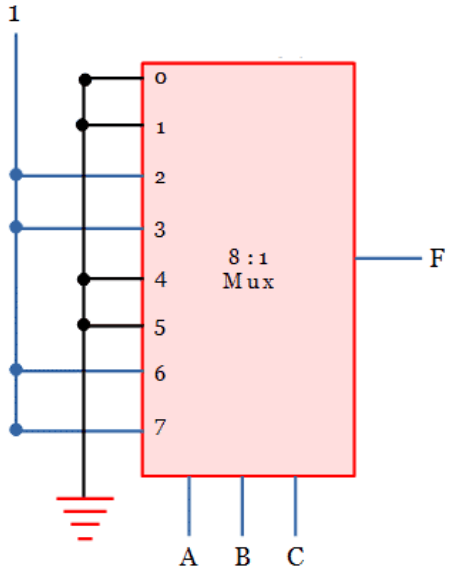
Example: Implement the Boolean function F (A, B, C, D) = Σ (0, 1, 3, 4, 8, 9, 15) with 8 to 1 multiplexer.

Solution:

	I_0	I_1	I_2	I_3	I_4	I_5	I_6	I_7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	1	0	A'	A'	0	0	A



Example: Implement the Boolean expression F (A, B, C) = Σ m (2, 3, 6, 7) using a multiplexer.
Solution: There are 3 variables in the given expression, hence $2^n = 2^3$ = 8:1 multiplexer. So, the mux has 8 input lines, 3 selection lines, and one output. The inputs, corresponding to the minterms (2, 3, 6, 7) are connected to the logic 1 and remaining terms to the logic 0 (grounded). The given input variables are connected as three selection lines.



SEQUENTIAL LOGIC CIRCUITS

INTRODUCTION:

Earlier we studied the digital circuits whose output at any instant of time are entirely dependent on the input present at that time. Such circuits are called as combinational circuits on the other hand **sequential circuits are those in which the output at any instant of time is determined by the applied input and past history of these inputs (i.e. present state).** Alternately, **sequential circuits are those in which output at any given time is not only dependent on the input, present at that time but also on previous outputs.** Naturally, such circuits must record the previous outputs. This gives rise to memory. Often, there are requirements of digital circuits whose output remain unchanged, once set, even if the inputs are removed. Such devices are referred as "memory elements", each of which can hold 1-bit of information. These binary bits can be retained in the memory indefinitely (as long as power is delivered) or until new information is feeded to the circuit.

A combinational logic circuit that consists of inputs variable (X), logic gates (Computational circuit), and output variable (Z).

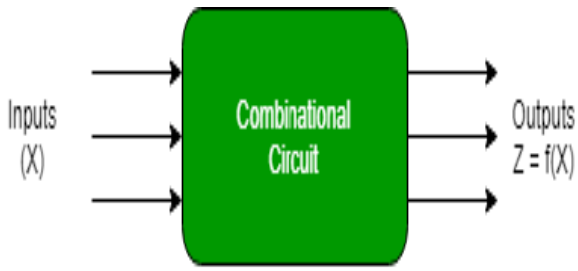


Figure: Block diagram of Combinational circuit

Combinational circuit produces an output based on input variable only, but **Sequential circuit** produces an output based on **current input and previous (past) input variables**. That means sequential circuits include memory elements which are capable of storing binary information. That binary information defines the state of the sequential circuit at that time. A latch capable of storing one bit of information.

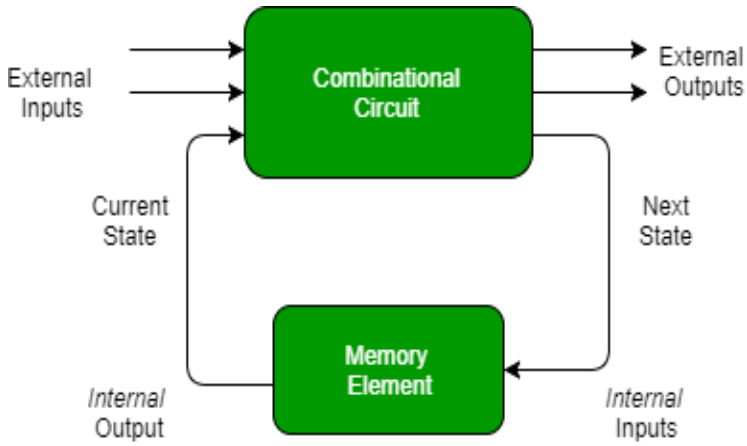


Figure: Block diagram of Sequential circuit

As shown in figure there are two types of input to the combinational logic:
1. External inputs which not controlled by the circuit.
2. Internal inputs which are a function of a previous output states.

Secondary inputs are state variables produced by the storage elements, whereas secondary outputs are excitations for the storage elements.

Combinational Circuits	Sequential Circuits
Outputs depend only on present inputs.	Outputs depend on both present inputs and present state.
Feedback path is not present.	Feedback path is present.
Memory elements are not required.	Memory elements are required.
Clock signal is not required.	Clock signal is required.
Easy to design.	Difficult to design.

Types of Sequential Circuits – There are two types of sequential circuit:

1. **Asynchronous sequential circuit –** These circuit **do not use a clock signal** but uses the pulses of the inputs. These circuits are **faster** than synchronous sequential circuits because there is clock pulse and change their state immediately when there is a change in the input signal. We use asynchronous sequential circuits when speed of operation is important and **independent** of internal clock pulse.

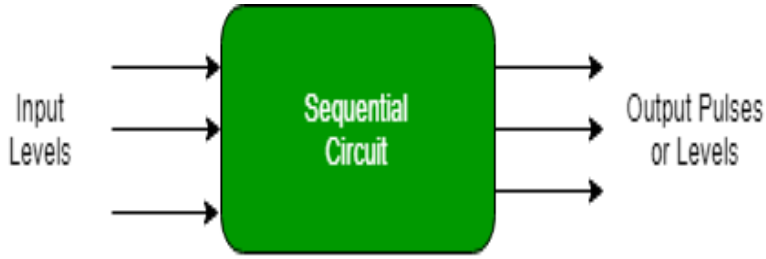


Figure: Asynchronous sequential circuit

But these circuits are more **difficult** to design and their output is **uncertain**.

2. Synchronous sequential circuit – These circuit **uses clock signal** and level inputs (or pulsed) (with restrictions on pulse width and circuit propagation). The output pulse is the same duration as the clock pulse for the clocked sequential circuits. Since they wait for the next clock pulse to arrive to perform the next operation, so these circuits are bit **slower** compared to asynchronous. Level output changes state at the start of an input pulse and remains in that until the next input or clock pulse.

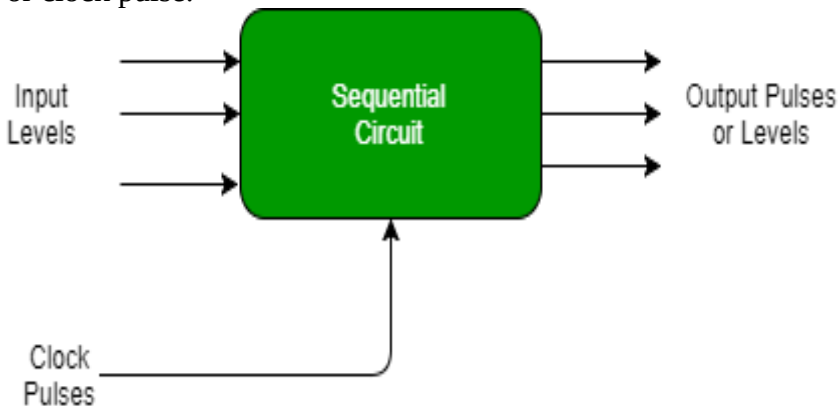


Figure: Synchronous sequential circuit

We use synchronous sequential circuit in synchronous counters, flip flops, and in the design of MOORE-MEALY state management machines. We use sequential circuits to design Counters, Registers, RAM, MOORE/MEALY Machine and other state retaining machines.

Synchronous sequential circuits	**Asynchronous sequential circuits**
In synchronous circuits, memory elements are clocked FFs.	In asynchronous circuits, memory elements are either un-clocked FFs or time delay elements.
The change in input signals can affect memory elements upon activation of clock signal.	The change in input signals can affect memory elements at any instant of time.
The maximum operating speed of the clock depends on time delays involved.	Because of the absence of the clock, asynchronous circuits can operate faster than synchronous circuits
Easier to design.	More difficult to design.

LATCHES AND FLIP-FLOPS:

There are two types of memory elements based on the type of triggering that is suitable to operate it.

• Latches
• Flip-flops

Differences between Flip flops and Latches:

Flip- flops	Latches
Flip flop is a bistable device, it has two states (0 and 1).	Latch is also a bistable device, it has two states (0 and 1).
It checks the inputs but changes the output only at times defined by the clock signal or other control signal.	It checks the inputs continuously and responds to changes in inputs immediately.
It is an edge triggered device.	It is a level triggered device.
Gates like NOR, NAND, NOT, AND are building blocks of flip flops.	These are also made up of logic gates
These are classified in to synchronous and asynchronous flip flops.	There is no such classification in latches.
These forms the building blocks of many sequential circuits like registers and counters.	These can be used for the designing of sequential circuits but are not generally preferred.
Always have a clock signal.	Doesn't have a clock signal.
Flip flops can be built from latches	Latches can be built from gates.
Ex: SR, D, JK, T- flip flops	Ex: SR, D, JK, T- Latches

Latches operate with enable signal, which is **level sensitive**. Whereas, **flip-flops are edge sensitive.**

Edge triggered Flip-Flops:

1. Positive Edge triggered Flip flop
2. Negative Edge triggered Flip flop

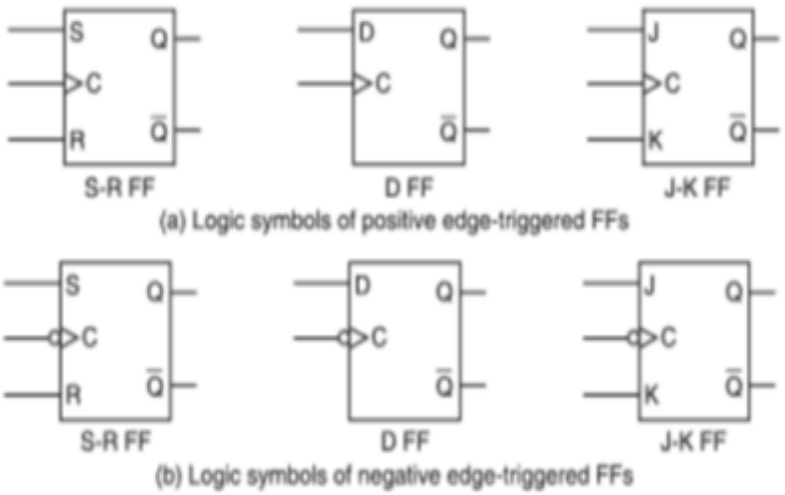


Figure: Logic symbols for positive and negative edge triggered flip-flops

POSITIVE EDGE TRIGGERED SR FLIP FLOP:

Case-1: Set = Reset = 0

This is the normal state of the FF and it has no effect on the output state. Q and Q' will remain in whatever they were prior to the occurrence of this input condition**.**

Case-2: Set = 0, Reset = 1

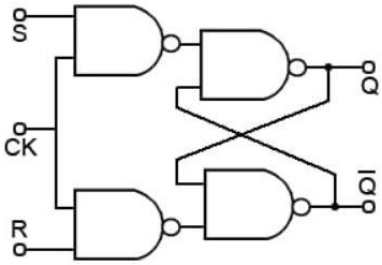
This will always set Q=0, where it will remain even after SET returns to 0, so Reset state occurs.

Case-3: Set = 1, Reset = 0

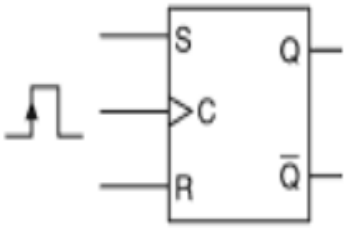
This will always set Q=1, where it will remain even after RESET returns to 1, so Set state occurs.

Case-4: Set = Reset = 1

This condition tries to SET and RESET the FF at the same time, and it produces Q = Q' =0. If the inputs are returned to zero simultaneously, the resulting output state is erratic and unpredictable. This input condition should not be used. It is forbidden.



(a) Logic diagram using NAND gates

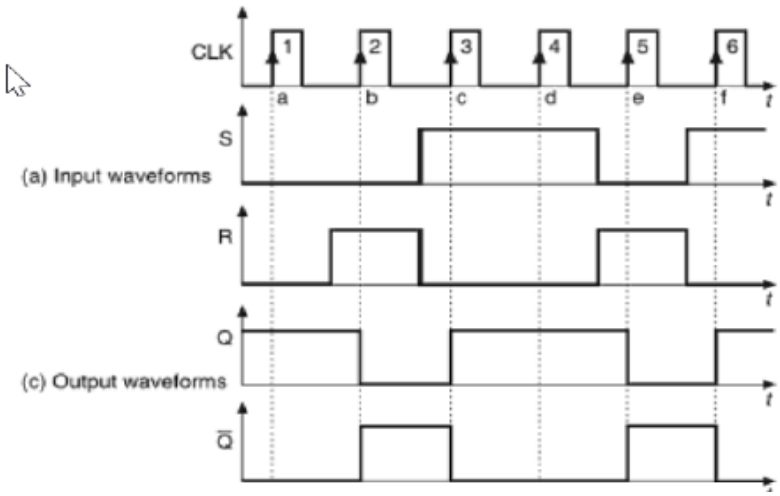


(b) Logic symbol

clk	S	R	Qn+1
0	×	×	Qn
↑	0	0	Qn
↑	0	1	0
↑	1	0	1
↑	1	1	×

clk	S	R	Qn	Qn+1	State
↑	0	0	0	0	No Change(NC)
↑	0	0	1	1	
↑	0	1	0	0	Reset
↑	0	1	1	0	
↑	1	0	0	1	Set
↑	1	0	1	1	
↑	1	1	0	×	Indeterminate (Invalid)
↑	1	1	1	×	
0	×	×	0	0	No Change(NC)
0	×	×	1	1	

(c) Truth table of positive edge triggered SR-FF



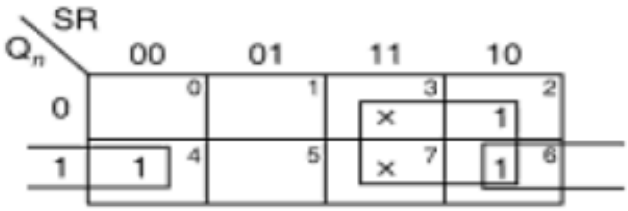
(d) Timing diagram of positive edge triggered SR- FF

Qn	S	R	Qn+1
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	X
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	X

(e) Characteristic table

Qn	Qn+1	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

(f) Excitation table



K-map for Q_{n+1} of SR flip-flop

The characteristic equation of SR flip-flop

$Q_{n+1} = S + Q_n\overline{R}$

(g) Characteristic equation

Figure: Positive edge triggered SR- Flip flop

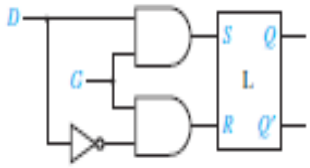
POSITIVE EDGE TRIGGERED D FLIP FLOP:

Case-1: D = 0

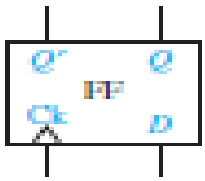
From this condition we get Qn+1' as 1. So the next state Qn+1 is 0. As the resultant is 0 it is known reset condition.

Case-2: D = 1

From this condition we get Qn+1 as 1. So this known as set condition.



(a) Logic diagram



(b) Logic symbol

clk	D	Qn	Qn+1	State
0	×	0	0	NC
0	×	1	1	
↑	0	0	0	Reset
↑	0	1	0	
↑	1	0	1	Set
↑	1	1	1	

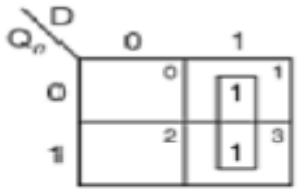
(c) Truth table

Qn	D	Qn+1
0	0	0
0	1	1
1	0	0
1	1	1

(d) Characteristic table

Qn	Qn+1	D
0	0	0
0	1	1
1	0	0
1	1	1

(e) Excitation table



K-map for Q_{n+1} of D flip-flop

The characteristic equation of D flip-flop is $Q_{n+1} = D$

(f) Characteristic equation

Figure: Positive edge triggered D- Flip flop

EDGE TRIGGERED JK FLIP FLOP:

Case 1: J = 0, K = 0

This is the normal state of the FF and it has no effect on the output state. Q and Not Q will remain in whatever they were prior to the occurrence of this input condition**.**

Case 2: J = 0, K = 1

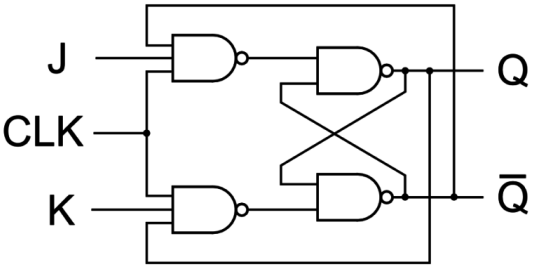
This will always set Q=0, where it will remain even after SET returns to 0

Case 3: J = 1, K = 0

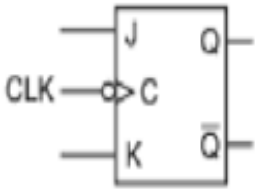
This will always Reset Q=1, where it will remain even after RESET returns to 0

Case 4: J = 1, K = 1

This condition is known as toggle condition. Since the process continues by getting alternating output i.e. frequently changing.



(a) Logic diagram



(b) Logic symbol (Negative edge triggered)

Clk	J	K	Qn	Qn+1	State
↑	0	0	0	0	No Change (NC)
↑	0	0	1	1	
↑	0	1	0	0	Reset
↑	0	1	1	0	
↑	1	0	0	1	Set
↑	1	0	1	1	
↑	1	1	0	1	Toggle
↑	1	1	1	0	
0	X	X	0	0	No Change
0	X	X	1	1	

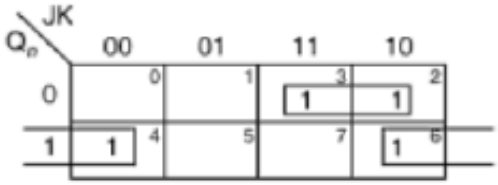
(c) Truth table

Qn	J	K	Qn+1
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

(d) Characteristic table

Qn	Qn+1	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

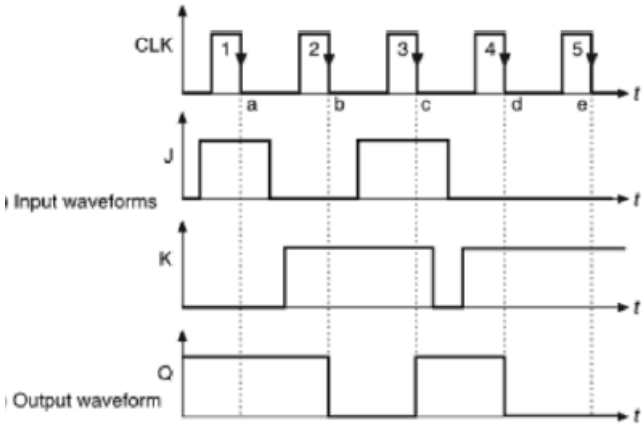
(e) Excitation table



The characteristic equation of a JK flip-flop is

$Q_{n+1} = \overline{Q}_n J + Q_n \overline{K}$

(f) Characteristic equation



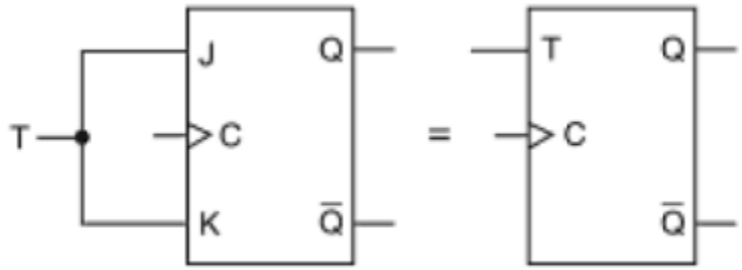
(g) Timing diagram of negative edge triggered JK- Flip flop

Figure: Edge triggered JK- Flip flop

EDGE TRIGGERED T FLIP FLOP:

Case 1: T = 0
The next state is equal to the present state. This is known as present state condition.

Case 2: T = 1
Here the output is complemented to the input. This is known as toggle condition.



(a) T flip-flop from JK flip-flop (b) Logic symbol

T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

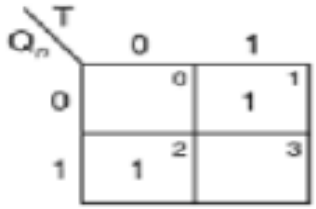
(c) Truth table

T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

(d) Characteristic table

Qn	Qn+1	T
0	0	0
0	1	1
1	0	1
1	1	0

(e) Excitation table



K-map for Q_{n+1} of T flip-flop

The characteristic equation of T flip-flop is

$$Q_{n+1} = \bar{Q}_n T + Q_n \bar{T}$$

(f) Characteristic equation

Characteristic equations for all the FF:

Flip-flop	Characteristic equation
D	$Q_{n+1} = D$
J-K	$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$
T	$Q_{n+1} = T\bar{Q}_n + \bar{T}Q_n$
S-R	$Q_{n+1} = S + \bar{R}Q_n$

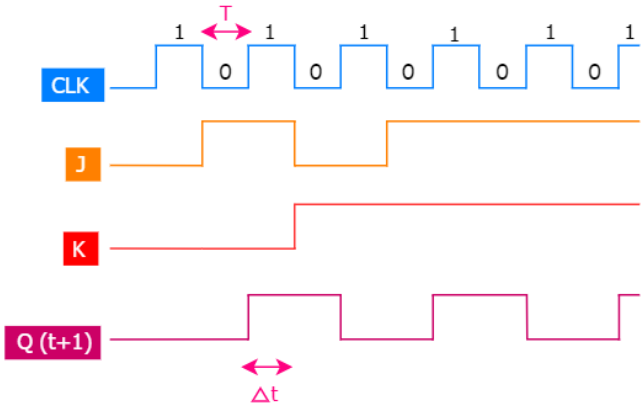
Race around Condition in JK Flip Flop:

Before getting into the race around condition, let us have a look at the JK flip-flop's truth table.

Clock Input	Inputs		Outputs		Comments
	J	K	Q	Q'	
0	X	X	Same as previous	Same as previous	No change
1	0	0	Same as previous	Same as previous	No change
1	0	1	0	1	Reset
1	1	0	1	0	Set
1	1	1	Opposite of previous	Opposite of previous	Toggle

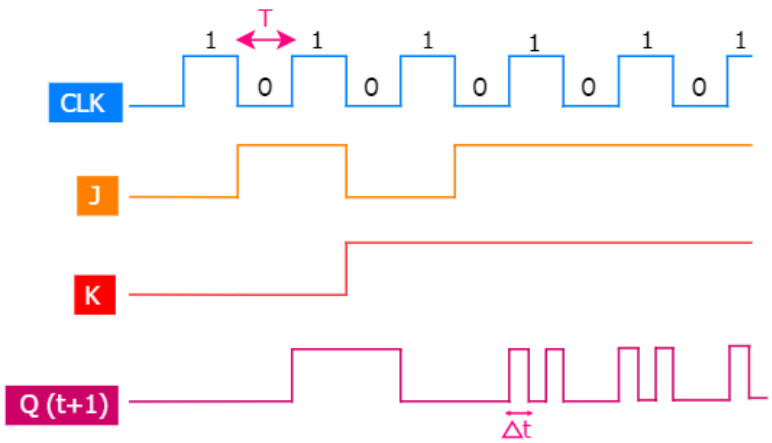
Here, Q is the present state and Q' is the next state. As you can see, when J, K and Clock are equal to 1, toggling takes place, i.e. the next state will be equal to the complement of the present state.

Now, let us look at the timing diagram of JK flip-flop.



Here, **T** is the **time period of the clock** whereas Δ **(Delta) t** is the **propagation delay**. The delay between input and output is called a propagation delay. This is what was expected, but the output may not be like this all the time. This is where **Race around condition** comes into the play.

Let us look at the timing diagram of JK flip-flop when the race around condition is considered.



As you already know, when J, K and Clock are equal to 1, toggling takes place. Here, propagation delay has also been reduced, so the output will be given out at the instant input is given. So there is a toggling again. Therefore, whenever Clock is equal to 1 there are consecutive toggling. This condition is called as **Race around condition**.

For JK flip-flop if J, K and Clock are equal to 1 the state of flip-flop keeps on toggling which leads to uncertainty in determining the output of the flip-flop. This problem is called Race around the condition. This condition also exists in T flip-flop since T flip-flop also has toggling options.

CONVERSION OF FLIP FLOPS:

The four flip-flops, namely SR flip-flop, D flip-flop, JK flip-flop & T flip-flop, can convert one flip-flop into the remaining three flip-flops by including some additional logic. So, there will be total of **twelve flip-flop conversions**.

Step 1: Identify the available and required FF.
Step 2: Make the characteristic Table for required Flip Flop.
Step 3: Make the Excitation table for available Flip Flop.
Step 4: Write Boolean Expression for available Flip Flop.
Step 5: Draw the Corresponding Logic Circuit.

1. SR FLIP FLOP TO JK FLIP FLOP:

Step 1: Identify the available and required FF.
Available→ SR Flip Flop
Required ----------→ JK Flip Flop

Step 2: Make the characteristic Table for required Flip Flop.

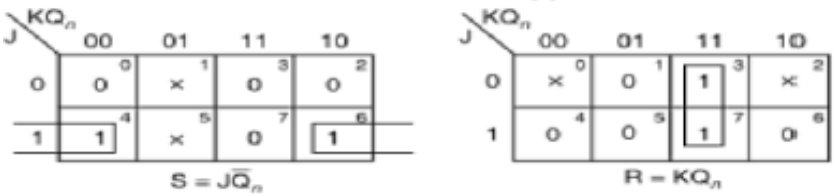
Qn	J	K	Qn+1
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Step 3: Make the Excitation table for available Flip Flop.

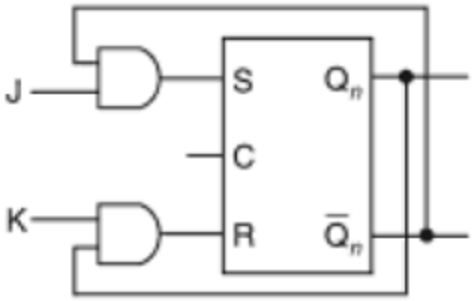
Qn	Qn+1	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Step 4: Write Boolean Expression for available Flip Flop.

External Inputs		Present State	Next State	Flip-flop Inputs	
J	K	Q_n	Q_{n+1}	S	R
0	0	0	0	0	×
0	0	1	1	×	0
0	1	0	0	0	×
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	×	0
1	1	0	1	1	0
1	1	1	0	0	1



Step 5: Draw the Corresponding Logic Circuit.



2. JK FLIP FLOP TO T FLIP FLOP:

Step 1: Identify the available and required FF.
Available → JK Flip Flop
Required → T Flip Flop

Step 2: Make the characteristic Table for required Flip Flop.

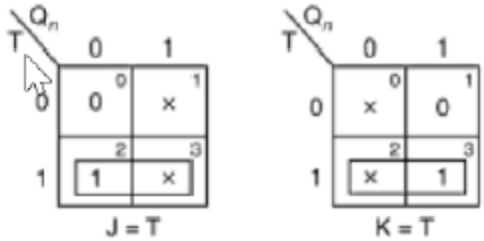
T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

Step 3: Make the Excitation table for available Flip Flop.

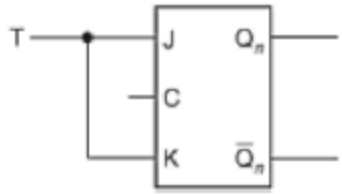
Qn	Qn+1	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Step 4: Write Boolean Expression for available Flip Flop.

External Input	Present State	Next State	Flip-flop Inputs	
T	Q_n	Q_{n+1}	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1



Step 5: Draw the Corresponding Logic Circuit.



SYNCHRONOUS AND ASYNCHRONOUS COUNTERS:

COUNTERS

Counter is a device which stores (and sometimes displays) the number of times particular event or process has occurred, often in relationship to a clock signal. A Digital counter is a set of flip flops whose state change in response to pulses applied at the input to the counter. Counters may be asynchronous counters or synchronous counters. **Asynchronous counters are also called ripple counters**.

In electronics, counters can be implemented quite easily using register-type circuits such as the flip-flops and a wide variety of classifications exist:

• **Asynchronous (ripple) counter** – changing state bits are used as clocks to subsequent state flip-flops.

• **Synchronous counter** – all state bits change under control of a single clock.

• **Decade counter** – counts through ten states per stage.

• **Up/down counter** – counts both up and down, under command of a control input

• **Ring counter** – formed by a shift register with feedback connection in a ring

• **Johnson counter** – a twisted ring counter

• Cascaded counter

• Modulus counter

SYNCHRONOUS COUNTER	ASYNCHRONOUS COUNTER
In synchronous counter, all flip flops are triggered with same clock simultaneously.	In asynchronous counter, different flip flops are triggered with different clock, not simultaneously.
Synchronous counter is faster than asynchronous counter in operation.	Asynchronous counter is slower than synchronous counter in operation.
Synchronous counter does not produce any decoding errors.	Asynchronous counter produces decoding error.
Synchronous counter is also called Parallel Counter.	Asynchronous counter is also called Serial Counter.
Synchronous counter designing as well implementation are complex due to increasing the number of states.	Asynchronous counter designing as well as implementation is very easy.
Synchronous counter will operate in any desired count sequence.	Asynchronous counter will operate only in fixed count sequence (UP/DOWN).
Synchronous counter examples are: Ring counter, Johnson counter.	Asynchronous counter examples are: Ripple UP counter, Ripple DOWN counter.
In synchronous counter, propagation delay is less.	In asynchronous counter, there is high propagation delay.

A counter may be an **up/down** counter.

- An **Up counter** will count numbers from small to high in up direction.
- And **Down counter** will count numbers from high to low in down direction.

Each of the counts of the counter is called the state of the counter.

The number of states through which the counter passes before returning to the starting state is called the **modulus counter**.

Each is useful for different applications. Usually, counter circuits are digital in nature, and count in natural binary Many types of counter circuits are available as digital building blocks, for example a number of chips in the 4000 series implement different counters.

Occasionally there are advantages to using a counting sequence other than the natural binary sequence such as the binary coded decimal counter, a linear feed-back shift register counter, or a Gray-code counter.

Counters are useful for digital clocks and timers, and in oven timers, VCR clocks, etc.

ASYNCHRONOUS COUNTERS:

An asynchronous (ripple) counter is a single JK-type flip-flop, with its J (data) input fed from its own inverted output. This circuit can store one bit, and hence can count from zero to one before it overflows (starts over from 0). This counter will increment once for every clock cycle and takes two clock cycles to overflow, so every cycle it will alternate between a transition from 0 to 1 and a transition from 1 to 0. Notice that this creates a new clock with a 50% duty cycle at exactly half the frequency of the input clock. If this output is then used as the clock signal for a similarly arranged D flip-flop (remembering to invert the output to the input), one will get another 1 bit counter that counts half as fast. Putting them together yields a two-bit counter.

TWO-BIT RIPPLE UP-COUNTER USING NEGATIVE EDGE TRIGGERED FLIP FLOP:

The two-bit ripple counter uses two flip-flops. There are four possible states from 2 – bit up- counting i.e. 00, 01, 10 and 11 (counting from 0, 1, 2, 3). For a flip flop we are having bubble at clock it means the negative edge triggered FF. In this First FF output Q1 is connected as clock to the next FF so it is called as **up counter using negative edge triggered.** The counter is initially assumed to be at a reset **state 00**.

When the first clock pulse is applied FF1 toggles at the negative –going edge of this pulse, therefore, Q1 goes from LOW to HIGH. This becomes a positive –going signal at the clock input of FF2 is not affected, and hence the state of the counter after one clock pulse is Q1=1 and Q2=0 i.e., **state 01**.

At the negative –going edge of the second clock pulse, FF1 toggles. So, Q1 goes from HIGH to LOW. This becomes a negative –going signal at the clock input of FF2 and hence Q2 goes from LOW to HIGH. Therefore, the state of the counter after second clock pulse is Q1=0 and Q2=1 i.e., **state 10**.

At the negative –going edge of the third clock pulse, FF1 toggles. So, Q1 goes from LOW to HIGH. This becomes a positive –going signal at the clock input of FF2 and is not affected. Therefore, the state of the counter after third clock pulse is Q1=1 and Q2=1 i.e., **state 11**.

At the negative –going edge of the fourth clock pulse, FF1 toggles. So, Q1 goes from HIGH to LOW HIGH. This becomes a negative –going signal at the clock input of FF2 and toggles. Therefore, the state of the counter after fourth clock pulse is Q1=0 and Q2=0 i.e., **state 00**.

The above process repeats between 00→01→10→11 →00 →01 →10 and so on. So it means that it is like mod 4 counter running from 0 to 3 and again from starting.

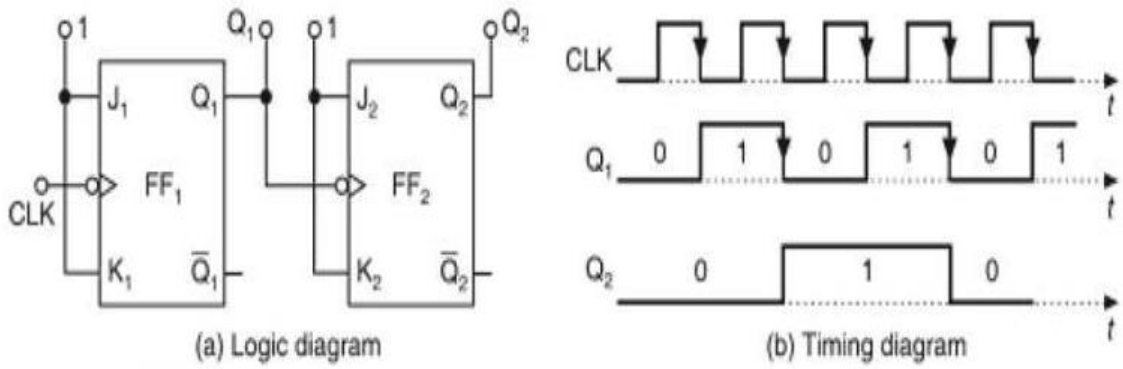


Figure: Asynchronous 2-bit ripple up-counter using negative edge triggered flip flop

TWO-BIT RIPPLE DOWN-COUNTER USING NEGATIVE EDGE TRIGGERED FLIP FLOP:

The two-bit ripple counter uses two flip-flops. There are four possible states from 2 – bit down- counting I.e. 11, 10, 01 and 00 (counting from 3, 2, 1, 0). For a flip flop we are having bubble at clock it means the negative edge triggered FF. In this First FF output Q1 is

connected as clock to the next FF so it is called as **down counter using negative edge triggered.** The counter is initially assumed to be at a reset **state 00**.

When the first clock pulse is applied FF1 toggles at the negative –going edge of this pulse, therefore, Q1 goes from LOW to HIGH and Q1 goes from a 1 to 0. This becomes a negative –going signal Q1 applied at the clock input of FF2 and toggles from 0 to a 1 and hence the state of the counter after one clock pulse is Q2=1 and Q1=1 i.e., **state 11**.

At the negative –going edge of the second clock pulse, FF1 toggles. So, Q1 goes from HIGH to LOW. Therefore, Q2 remains at 1. Hence, the state of the counter after second clock pulse is **state10**.

At the negative –going edge of the third clock pulse, FF1 toggles. So, Q1 goes from LOW to HIGH. And Q1 from a 1 to a 0. This becomes a negative –going signal at the clock input of FF2 so Q2 changes from a 1 to 0. Therefore, the state of the counter after third clock pulse is Q1=1 and Q2=0 i.e., **state 01**.

At the negative –going edge of the fourth clock pulse, FF1 toggles. So, Q1 goes from HIGH to LOW HIGH and Q1 from a 0 to a 1. This becomes a positive –going signal at the clock input of FF2 is not affected. Therefore, the state of the counter after fourth clock pulse is Q1=0 and Q2=0 i.e., **state 00**.

The above process repeats between 00 → 11 → 10 → 01→ 00 → 11 →10 so on. So it means that it is like mod 4 counter running from 3 to 0 and again from starting.

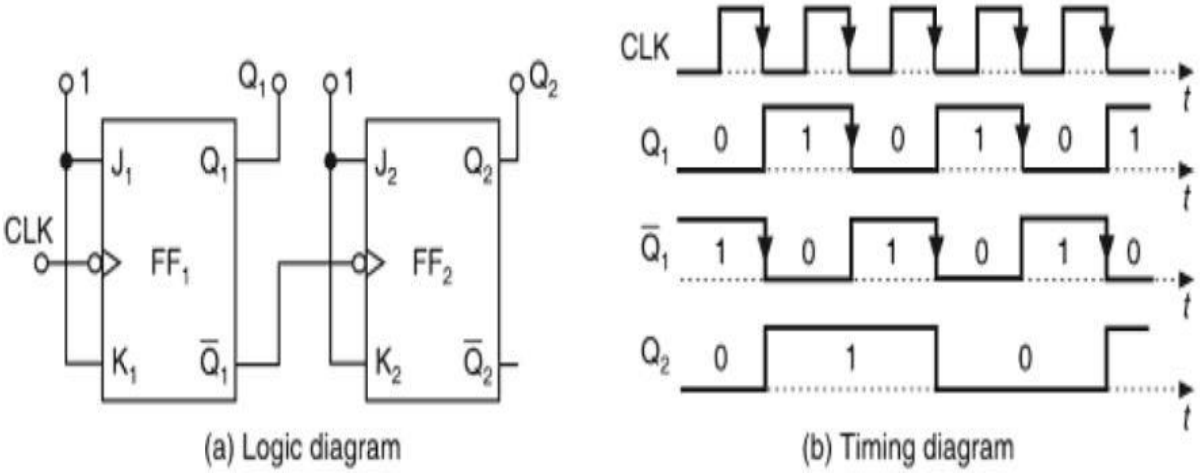


Figure: Asynchronous 2-bit ripple down-counter using negative edge triggered flip flop

TWO-BIT RIPPLE UP-DOWN COUNTER USING NEGATIVE EDGE TRIGGERED FLIP FLOP:

As the name indicates an up-down counter is a counter which can count both in upward and downward directions. An up-down counter is also called a forward/backward counter or a bidirectional counter. So, a control signal or a mode signal M is required to choose the direction of count. When M=1 for up counting, Q1 is transmitted to clock of FF2 and when M=0 for down counting, Q1' is transmitted to clock of FF2. This is achieved by using two AND gates and one OR gates. The external clock signal is applied to FF1.

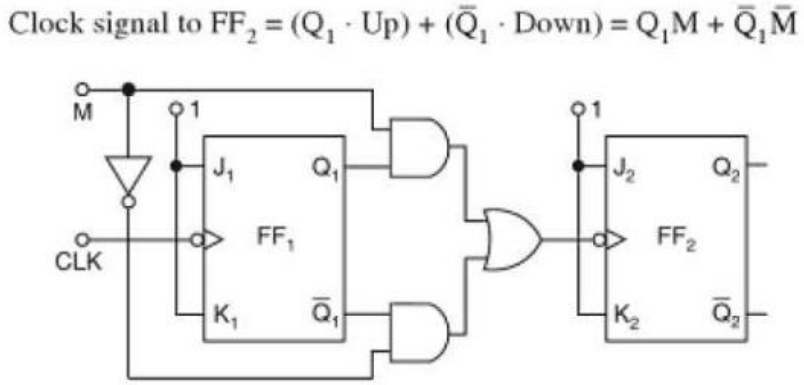


Figure: Asynchronous 2-bit ripple up-down counter using negative edge triggered flip flop

Two-bit ripple up-counter using positive edge triggered flip flop:

The two-bit ripple counter uses two flip-flops. There are four possible states from 2 – bit up- counting i.e. 00, 01, 10 and 11 (counting from 0, 1, 2, 3). For a flip flop we are not having bubble at clock it means the positive edge triggered FF.

In this First FF output Q1 is connected as clock to the next FF so it is called as **up counter using positive edge triggered.** The counter is initially assumed to be at a reset **state 00**. FF1 toggles at the positive going edge of each clock pulse and FF2 toggles whenever Q1' changes from a 0 to a 1. State transitions occur at the positive – going edges of the clock pulses. The counting sequence is 00, 01, 10, 11, 00, 01, 10, 11 etc.

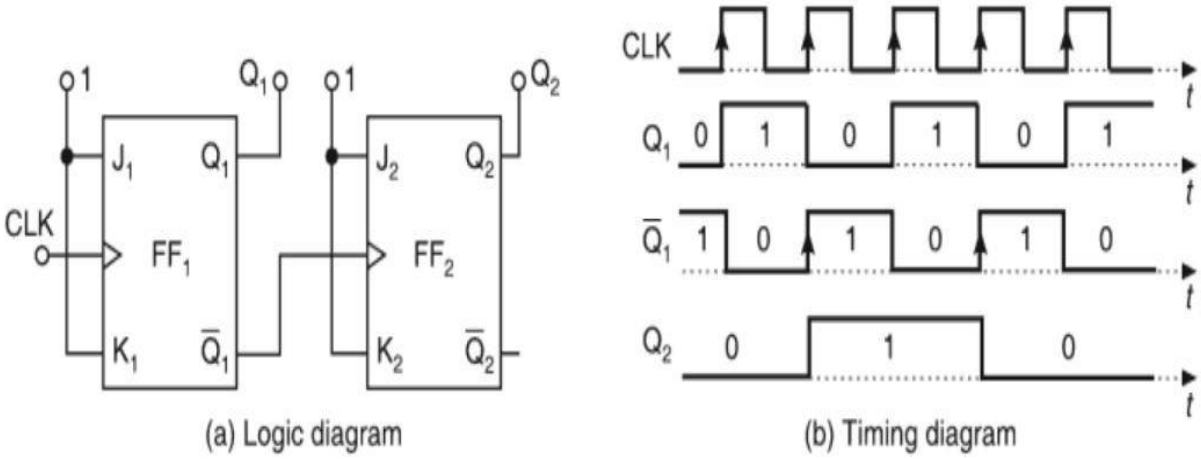


Figure: Asynchronous 2-bit ripple up-counter using positive edge triggered flip flop

Two-bit ripple down-counter using positive edge triggered flip flop:

The two-bit ripple counter uses two flip-flops. There are four possible states from 2 – bit down- counting I.e. 11, 10, 01 and 00 (counting from 3, 2, 1, 0). For a flip flop we are not having bubble at clock it means the positive edge triggered FF.

In this First FF output Q1 is connected as clock to the next FF so it is called as **down counter using positive edge triggered.** The counter is initially assumed to be at a reset **state 00**. FF1 toggles at the positive going edge of each clock pulse and FF2 toggles whenever Q1' changes from a 0 to a 1. State transitions occur at the positive – going edges of the clock pulses. The counting sequence is 00, 11, 10, 01, 00, 11, 10, 11 etc.

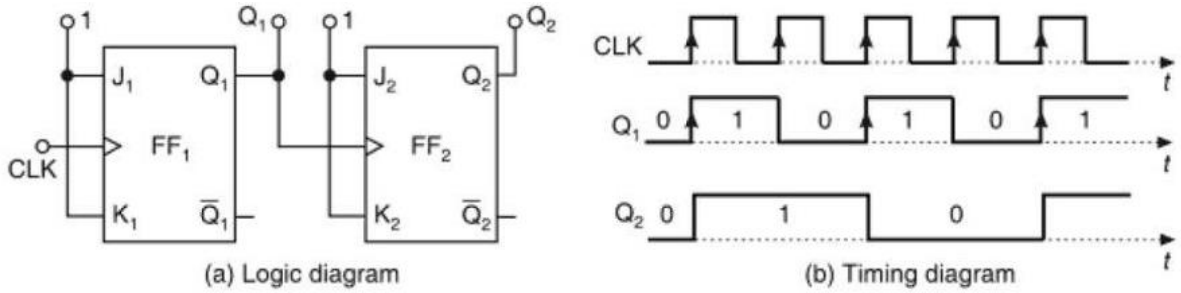


Figure: Asynchronous 2-bit ripple down-counter using positive edge triggered flip flop

Two-bit ripple up-down counter using positive edge triggered flip flop:

As the name indicates an up-down counter is a counter which can count both in upward and downward directions. An up-down counter is also called a forward/backward counter or a bidirectional counter. So, a control signal or a mode signal M is required to choose the direction of count. When M=1 for up counting, Q1' is transmitted to clock of FF2 and when M=0 for down counting, Q1 is transmitted to clock of FF2. This is achieved by using two AND gates and one OR gates. The external clock signal is applied to FF1.

$$\text{Clock signal to } FF_2 = (\bar{Q}_1 \cdot \text{Up}) + (Q_1 \cdot \text{Down}) = \bar{Q}_1 M + Q_1 \bar{M}$$

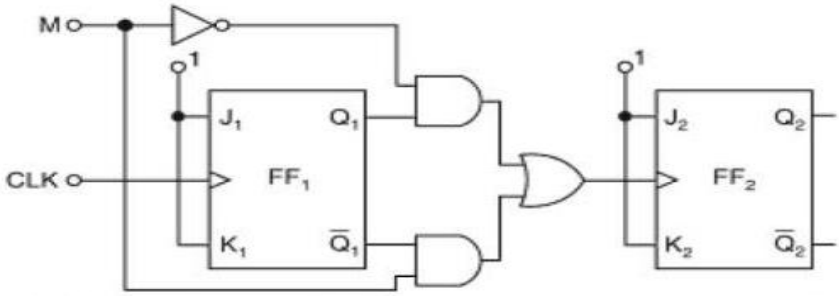


Figure: Asynchronous 2-bit ripple up-down counter using positive edge triggered flip flop

DESIGN OF ASYNCHRONOUS COUNTERS:

To design an asynchronous counter, first we write the sequence, then tabulate the values of reset signal R for various states of the counter and obtain the minimal expression for R and R' using K-Map or any other method. Provide a feedback such that R and R' resets all the FF's after the desired count.

DESIGN OF A MOD-6 ASYNCHRONOUS COUNTER USING T FFS:

A mod-6 counter has six stable states 000, 001, 010, 011, 100, and 101. When the sixth clock pulse is applied, the counter temporarily goes to 110 state, but immediately resets to 000 because of the feedback provided. It is —divide by-6-counter, in the sense that it divides the input clock frequency by 6. It requires three FFs, because the smallest value of n satisfying the condition $N \leq 2^n$ is n=3; three FFs can have 8 possible states, out of which only six are utilized and the remaining two states 110 and 111, are invalid. If initially the counter is in 000 state, then after the sixth clock pulse, it goes to 001, after the second clock pulse, it goes to 010, and so on.

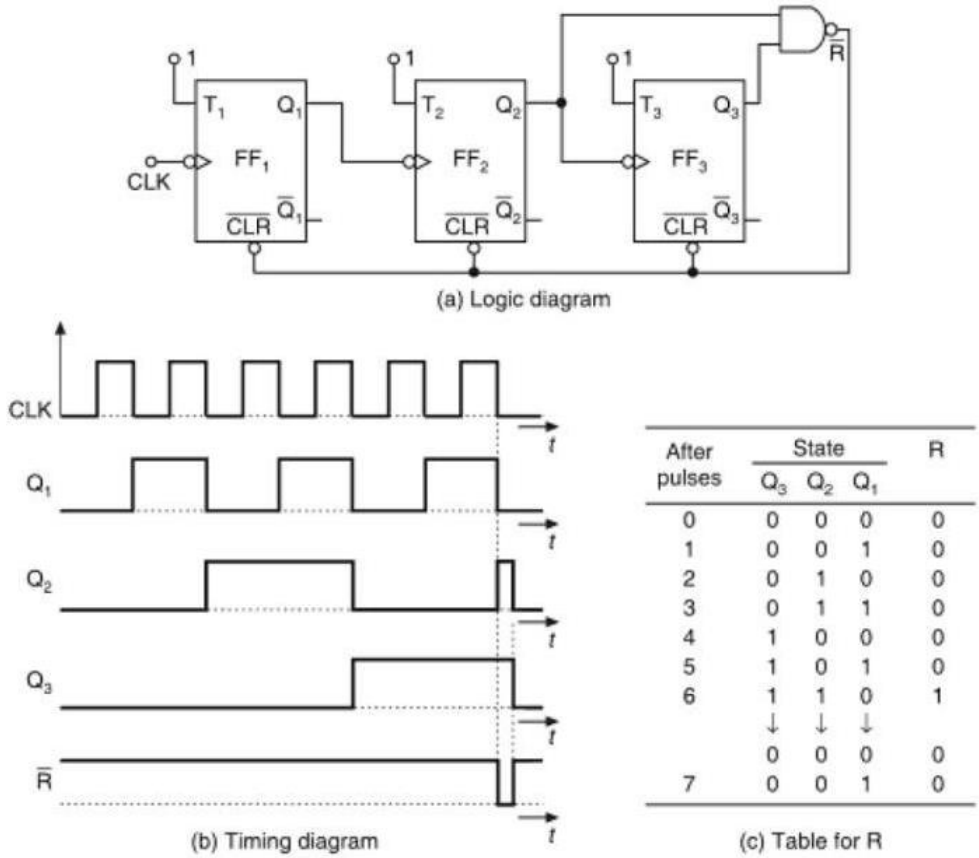
After **s**ixth clock pulse it goes to 000. For the design, write the truth table with present state outputs Q3, Q2 and Q1 as the variables, and reset R as the output and obtain an expression

for R in terms of Q3, Q2, and Q1that decides the feedback into be provided. From the truth table, R=Q3Q2. For active-low Reset, R' is used. The reset pulse is of very short duration, of the order of nanoseconds and it is equal to the propagation delay time of the NAND gate used.

The expression for R can also be determined as follows.
Therefore,

R= 0 for 000 to 101, R=1 for 110, and R =X = for111
R = Q3Q2Q1'+ Q3Q2Q1 = Q3Q2

The logic diagram, timing diagram and truth table of Mod-6 counter is shown in the below figure.



After pulses	State Q_3	Q_2	Q_1	R
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	0
5	1	0	1	0
6	1	1	0	1
	↓	↓	↓	
	0	0	0	0
7	0	0	1	0

Figure: Asynchronous Mod-6 counter using T- Flip flops

DESIGN OF A MOD-10 ASYNCHRONOUS COUNTER USING T-FLIP-FLOPS:

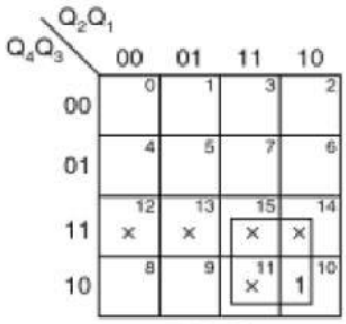
A mod-10 counter is a decade counter. It also called a BCD counter or a divide-by-10 counter. It requires four flip-flops (condition $10 \leq 2^n$ is n = 4). So, there are 16 possible states, out of which ten are valid and remaining six are invalid. The counter has ten stable state, 0000 through 1001, i.e., it counts from 0 to 9. The initial state is 0000 and after nine clock pulses it goes to 1001. When the tenth clock pulse is applied, the counter goes to state 1010 temporarily, but because of the feedback provided, it resets to initial state 0000. So, there will be a glitch in the waveform of Q2. The state 1010 is a temporary state for which the reset signal R=1, R=0 for 0000 to 1001, and R = C for 1011 to 1111.

The count table and the K-Map for reset are shown in below figure. From the K-Map R = Q4Q2. So, feedback is provided from second and fourth FFs. For active –HIGH reset,

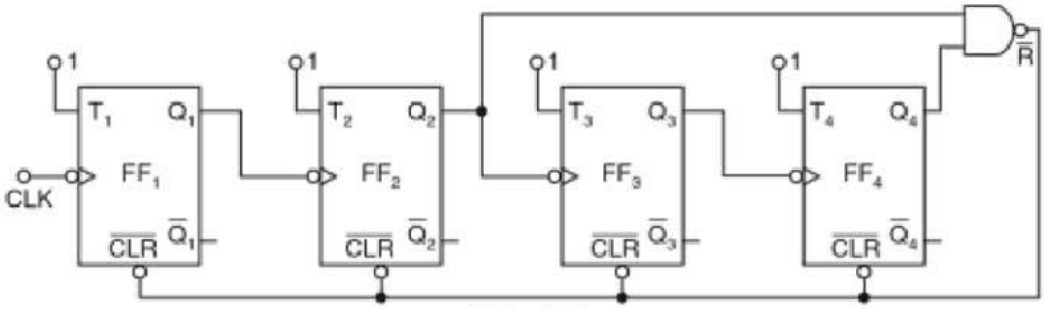
Q4Q2 is applied to the clear terminal. For active-LOW reset Q4Q2 is connected is of all Flip-flops.

After pulses	Count Q_4	Q_3	Q_2	Q_1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	0	0	0	0

(a) Count table



(b) K-Map



(c) Logic diagram

Figure: Asynchronous Mod-10 counter using T- Flip flops

SYNCHRONOUS COUNTERS:

Asynchronous counters are serial counters. They are slow because each FF can change state only if all the preceding FFs have changed their state. If the clock frequency is very high, the asynchronous counter may skip some of the states. This problem is overcome in **synchronous counters or parallel counters**. **Synchronous counters are counters in which all the flip flops are triggered simultaneously by the clock pulses**. Synchronous counters have a common clock pulse applied simultaneously to all flip-flop.

Design of synchronous counters:

For a systematic design of synchronous counters. The following procedure is used.

Step 1: Number of Flip-flops: Based on the description of the problem, determine the required number n of the flip-flops, the smallest value of n is such that the number of states $N \leq 2^n$ and the desired counting sequence.

Step 2: State Diagram: Draw the state diagram showing all the possible states state diagram which also be called nth transition diagrams, is a graphical means of depicting the sequence of states through which the counter progresses.

Step 3: Choice of Flip-flops excitation table: Select the type of flip-flop to be used and write the excitation table. An excitation table is a table that lists the present state (PS), the next state (NS) and required excitations.

Step 4: **Minimal expressions for excitations:** Obtain the minimal expressions for the excitations of the FF using K-maps drawn for the excitation of the flip-flops in terms of the present states and inputs.

Step 5: **Logic diagram**: Draw a logic diagram based on the minimal expressions.

Table Excitation tables

PS	NS	Required inputs	
Q_n	Q_{n+1}	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0

(a) S-R FF excitation table

PS	NS	Required inputs	
Q_n	Q_{n+1}	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0

(b) J-K FF excitation table

PS	NS	Required input
Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

(c) D FF excitation table

PS	NS	Required input
Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

(d) T FF excitation table

Figure: Excitation of various Flip-flops

DESIGN OF SYNCHRONOUS 3-BIT UP COUNTER:

Step 1: Determine the number of flip-flops required: A 3-bit up counter requires three FFs. It has 8 states (000, 001, 010, 011, 100, 101, 110, 111, and 000) and all the states are valid.

Step 2: Draw the state diagrams: The state diagram of the 3-bit up counter is drawn as shown in the below figure.

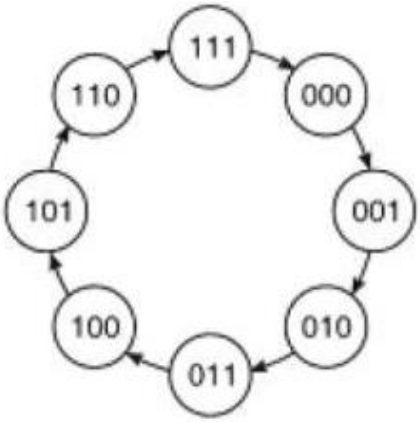


Figure: State diagram of synchronous 3-bit up-counter

Step 3: Select the type of flip flop and draw the excitation table: JK flip-flops are selected and the excitation table of a 3-bit up- counter using JK flip-flops is drawn as shown in the below figure.

PS			NS			Required excitations					
Q_3	Q_2	Q_1	Q_3	Q_2	Q_1	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	0	0	1	0	×	0	×	1	×
0	0	1	0	1	0	0	×	1	×	×	1
0	1	0	0	1	1	0	×	×	0	1	×
0	1	1	1	0	0	1	×	×	1	×	1
1	0	0	1	0	1	×	0	0	×	1	×
1	0	1	1	1	0	×	0	1	×	×	1
1	1	0	1	1	1	×	0	×	0	1	×
1	1	1	0	0	0	×	1	×	1	×	1

Figure: Excitation table of synchronous 3-bit up-counter

Step 4: Obtain the minimal expressions: From the excitation table we can conclude that J1=1 and K1=1, because all the entries for J1and K1 are either X or 1. The K-maps for J3, K3, J2 and K2 based on the excitation table and the minimal expression obtained from them are shown in the below figure.

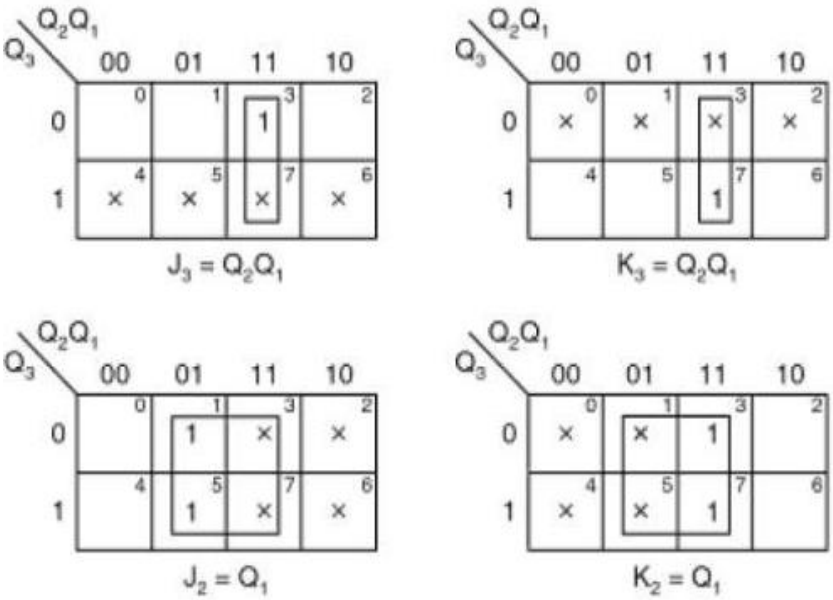


Figure: K-map minimization of synchronous 3-bit up-counter

Step 5: Draw the logic diagram: The logic diagram using those minimal expressions can be drawn as shown in the below figure.

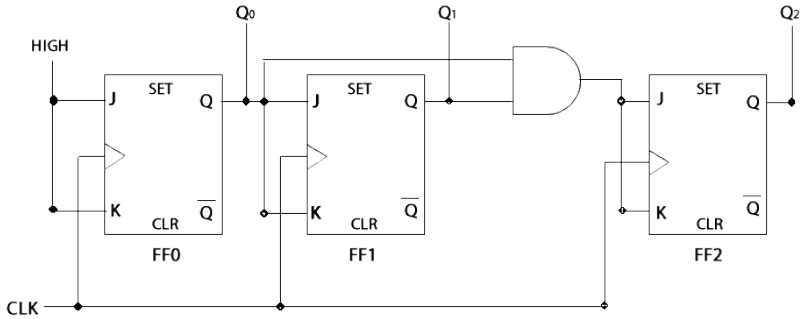


Figure: Logic diagram of synchronous 3-bit up counter

DESIGN OF SYNCHRONOUS 3-BIT DOWN COUNTER:

Step 1: Determine the number of flip-flops required: A 3-bit down counter requires three FFs. It has 8 states (000, 111, 110, 101, 100, 011, 010, 001, 000) and all the states are valid.

Step 2: Draw the state diagrams: the state diagram of the 3-bit down counter is drawn as shown in the below figure.

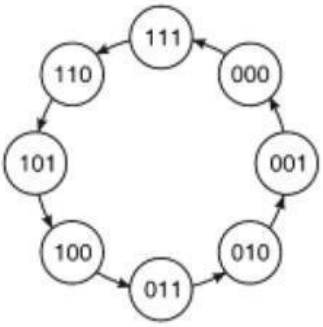


Figure: State diagram of synchronous 3-bit down counter

Step 3: Select the type of flip flop and draw the excitation table: JK flip-flops are selected and the excitation table of a 3-bit down- counter using JK flip-flops is drawn as shown in the below figure.

PS			NS			Required excitations					
Q_3	Q_2	Q_1	Q_3	Q_2	Q_1	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	1	1	1	1	×	1	×	1	×
1	1	1	1	1	0	×	0	×	0	×	1
1	1	0	1	0	1	×	0	×	1	1	×
1	0	1	1	0	0	×	0	0	×	×	1
1	0	0	0	1	1	×	1	1	×	1	×
0	1	1	0	1	0	0	×	×	0	×	1
0	1	0	0	0	1	0	×	×	1	1	×
0	0	1	0	0	0	0	×	0	×	×	1

Figure: K-map minimization of synchronous 3-bit down counter

Step 4: Obtain the minimal expressions: From the excitation table we can conclude that J1=1 and K1=1, because all the entries for J1and K1 are either X or 1. The K-maps for J3, K3, J2 and K2 based on the excitation table and the minimal expression obtained from them are shown in the below figure.

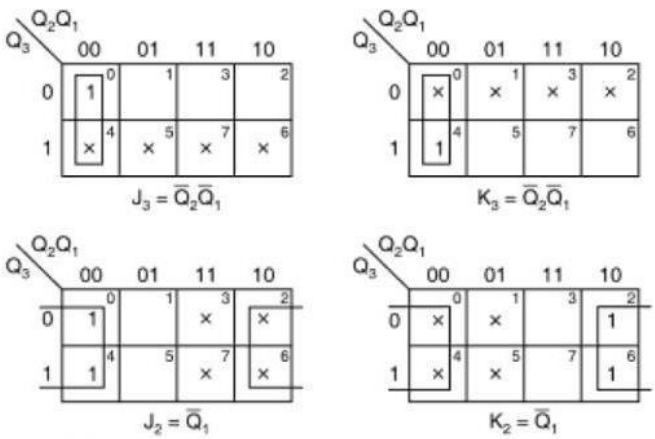
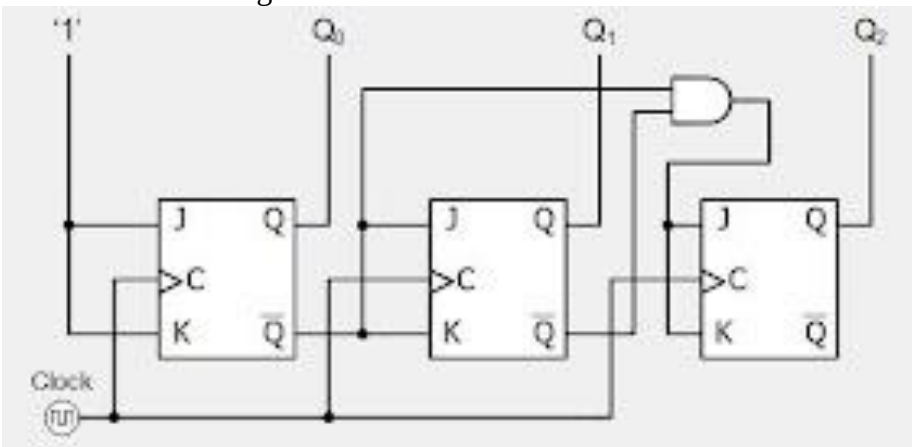


Figure: K-map minimization of synchronous 3-bit down counter

Step 5: Draw the logic diagram: A logic diagram using those minimal expressions can be drawn as shown in the below figure.



DESIGN OF A SYNCHRONOUS 3-BIT UP-DOWN COUNTER USING JK FLIP-FLOPS:

Step 1: Determine the number of flip-flops required: A 3-bit counter requires three FFs. It has 8 states (000,001,010,011,101,110,111) and all the states are valid. Hence no don‘t cares. For selecting up and down modes, a control or mode signal M is required. When the mode signal M=1 and counts down when M=0. The clock signal is applied to all the FFs simultaneously.

Step 2: Draw the state diagrams: The state diagram of the 3-bit up-down counter is drawn as shown in figure.

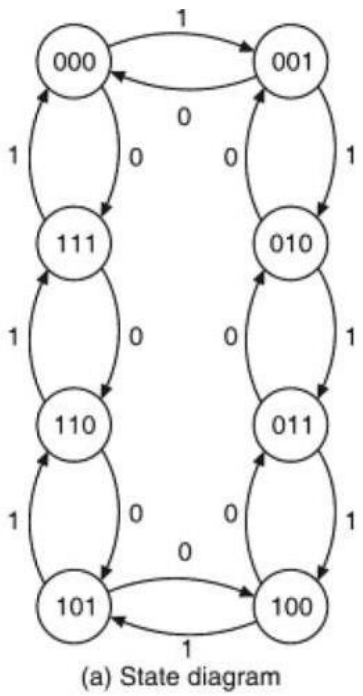


Figure: State diagram of synchronous 3-bit up-down counter

Step 3: Select the type of flip flop and draw the excitation table: JK flip-flops are selected and the excitation table of a 3-bit up-down counter using JK flip-flops is drawn as shown in figure.

PS			Mode	NS			Required excitations					
Q_3	Q_2	Q_1	M	Q_3	Q_2	Q_1	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	0	1	1	1	1	×	1	×	1	×
0	0	0	1	0	0	1	0	×	0	×	1	×
0	0	1	0	0	0	0	0	×	0	×	×	1
0	0	1	1	0	1	0	0	×	1	×	×	1
0	1	0	0	0	0	1	0	×	×	1	1	×
0	1	0	1	0	1	1	0	×	×	0	1	×
0	1	1	0	0	1	0	0	×	×	0	×	1
0	1	1	1	1	0	0	1	×	×	1	×	1
1	0	0	0	0	1	1	×	1	1	×	1	×
1	0	0	1	1	0	1	×	0	0	×	1	×
1	0	1	0	1	0	0	×	0	0	×	×	1
1	0	1	1	1	1	0	×	0	1	×	×	1
1	1	0	0	1	0	1	×	0	×	1	1	×
1	1	0	1	1	1	1	×	0	×	0	1	×
1	1	1	0	1	1	0	×	0	×	0	×	1
1	1	1	1	0	0	0	×	1	×	1	×	1

(b) Excitation table

Figure: Excitation table of synchronous 3-bit up-down counter

Step 4: Obtain the minimal expressions: From the excitation table we can conclude that J1=1 and K1=1, because all the entries for J1and K1 are either X or 1. The K-maps for J3, K3, J2 and K2 based on the excitation table and the minimal expression obtained from them are shown in below figure.

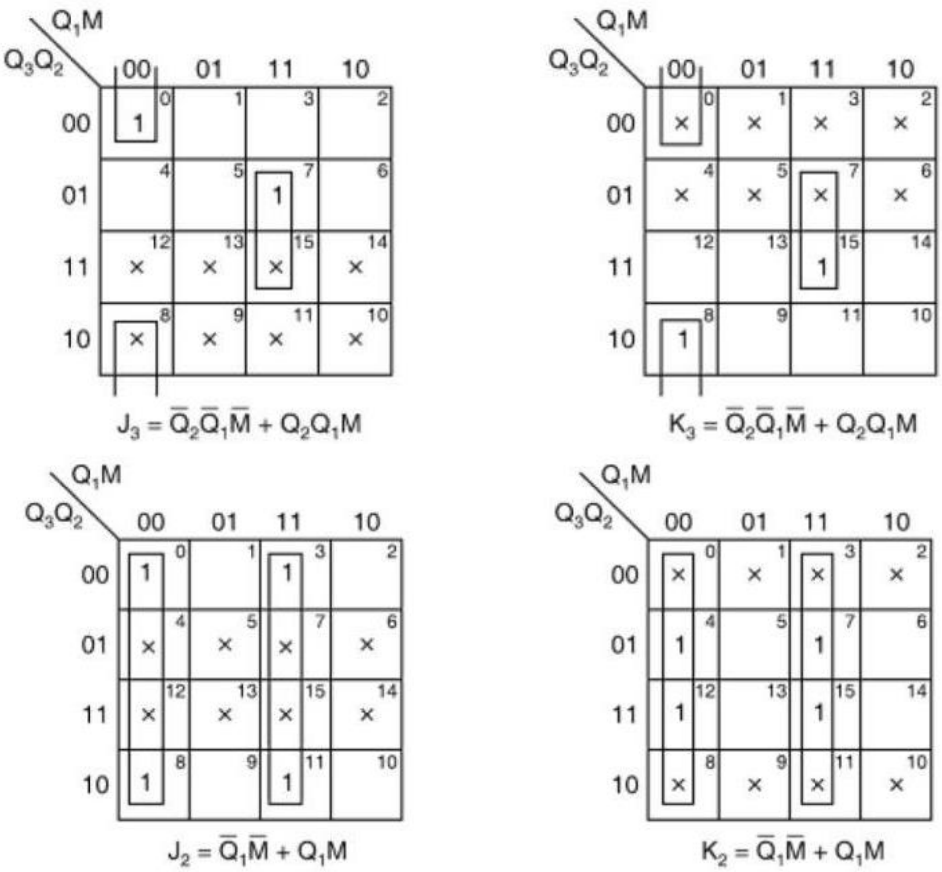


Figure: K-map minimization of synchronous 3-bit up-down counter

Step 5: Draw the logic diagram: Logic diagram using those minimal expressions can be drawn as shown in figure.

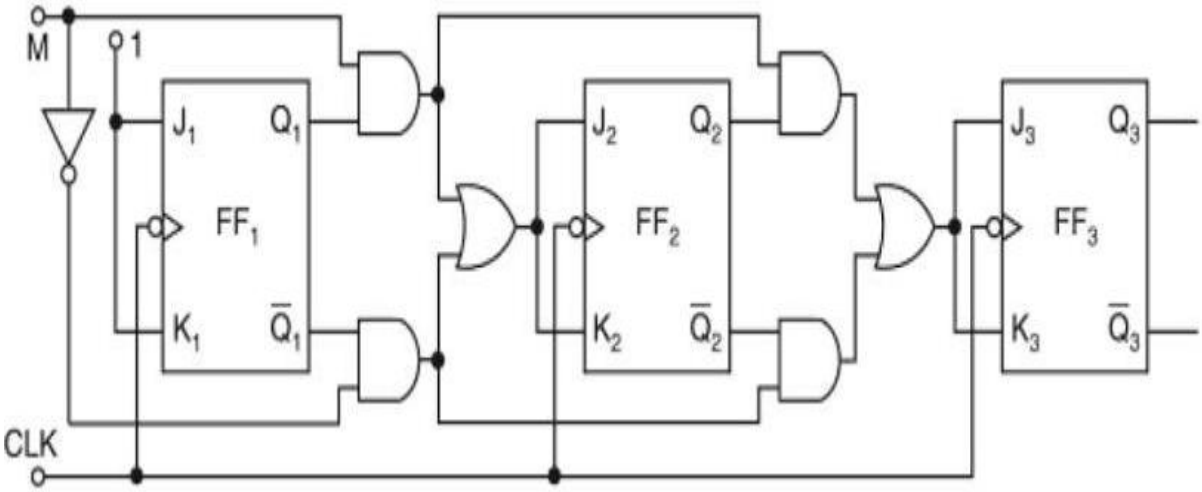


Figure: Logic diagram of synchronous 3-bit up-down counter

DESIGN OF SYNCHRONOUS MODULO-10 UP/DOWN COUNTER USING T FFS:

Step 1: Determine the number of flip-flops required: A modulo-10 has 10 states and so it requires 4 FFs. ($10 \leq 2^4$).

4 FF can have 16 states. So out of 16, six states are invalid (1010 through 1111).

M=1 → Up Counter

M=0 → Down Counter

Step 2: Draw the state diagrams: The state diagram of the mod-10 up/down counter is drawn as shown in the below figure.

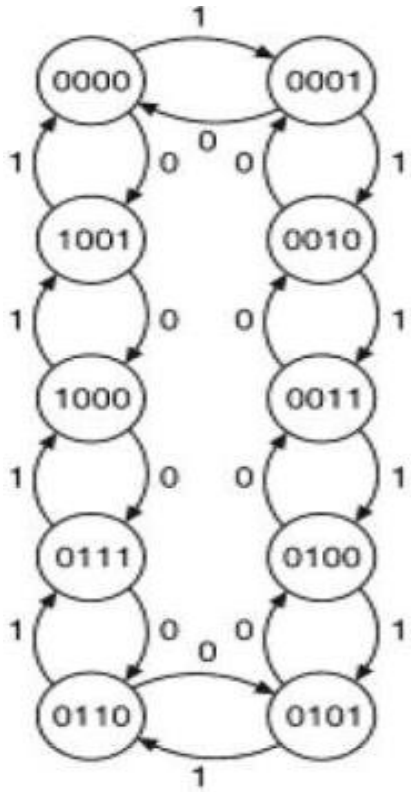


Figure: State diagram of synchronous mod-10 up-down counter

Step 3: Select the type of flip flop and draw the excitation table: T flip-flops are selected and the excitation table of modulo 10 up/down- counter using T flip-flops is drawn as shown in the below figure.

PS				Mode	NS				Required excitations			
Q_4	Q_3	Q_2	Q_1	M	Q_4	Q_3	Q_2	Q_1	T_4	T_3	T_2	T_1
0	0	0	0	0	1	0	0	1	1	0	0	1
0	0	0	0	1	0	0	0	1	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	1
0	0	0	1	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	0	0	1	0	0	1	1
0	0	1	0	1	0	0	1	1	0	0	0	1
0	0	1	1	0	0	0	1	0	0	0	0	1
0	0	1	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	0	0	1	1	0	1	1	1
0	1	0	0	1	0	1	0	1	0	0	0	1
0	1	0	1	0	0	1	0	0	0	0	0	1
0	1	0	1	1	0	1	1	0	0	0	1	1
0	1	1	0	0	0	1	0	1	0	0	1	1
0	1	1	0	1	0	1	1	1	0	0	0	1
0	1	1	1	0	0	1	1	0	0	0	0	1
0	1	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	0	1	0	0	0	1
1	0	0	1	0	1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	0	0	1	0	0	1

Figure: Excitation table of synchronous mod-10 up-down counter

Step 4: Obtain the minimal expressions: From the excitation table we can conclude that T=1, because all the entries for T1 are 1. The K-maps T4, T3 and T2 based on the excitation table. In the K-maps, the remaining minterms are don't cares (Σd (20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31))

From the excitation table we can see that T1 = 1 and the expression for T4, T3, and T2 are

T4 = Σm (0, 15, 16, 19) + d (20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31)

T3 = Σm (7, 8, 15, 16) + d (20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31)

T2 = Σm (3, 4, 7, 8, 11, 12, 15, 16) + d (20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31)

The minimal expression for T2 obtained by minimizing the K-map is given below.

$$\mathbf{T2 = Q4\overline{Q1}\overline{M} + \overline{Q4}Q1M + Q2\overline{Q1}M + Q3\overline{Q1}M}$$

Step5: Draw the logic diagram: A logic diagram using those minimal expressions can be drawn as shown in the below figure.

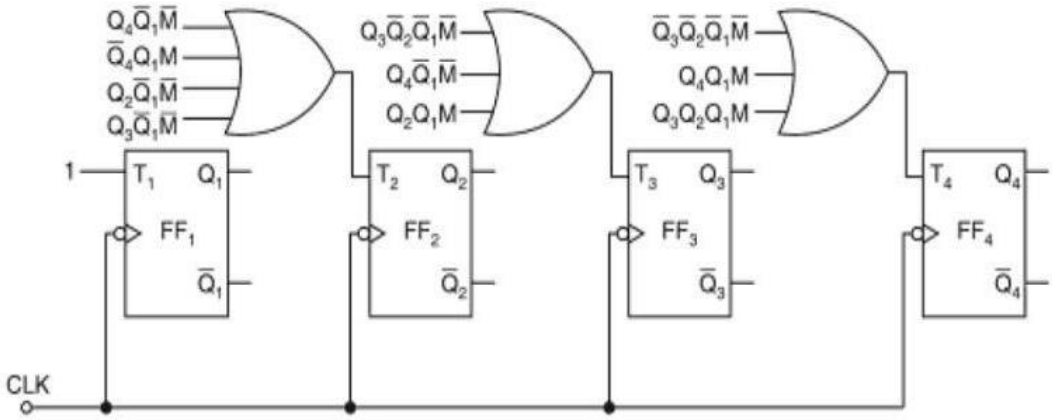


Figure: Logic diagram of synchronous mod-10 up-down counter

DESIGN OF MODULO-9 SYNCHRONOUS COUNTER USING T FFS:

Step 1: Determine the number of flip-flops required: A modulo 9 counter requires four FFs. It has 9 states (0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 0000……) all the states are valid. There are invalid from 1001 to 1111.

Step 2: Draw the state diagrams: the state diagram of the mod-9 counter is drawn as shown in the below figure.

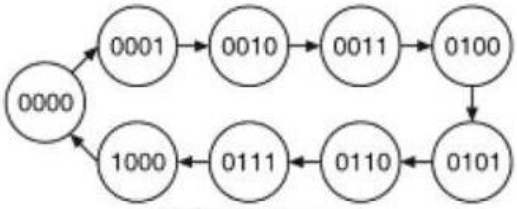


Figure: State diagram of synchronous modulo-9 counter

Step 3: Select the type of flip flop and draw the excitation table: T flip-flops are selected and the excitation table of a mod 9 counter using T flip-flops is drawn as shown in the below figure.

PS				NS				Required Excitations			
Q_4	Q_3	Q_2	Q_1	Q_4	Q_3	Q_2	Q_1	T_4	T_3	T_2	T_1
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	1
0	0	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	1
0	1	0	1	0	1	1	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	1
0	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	1	0	0	0

Figure: Excitation table of synchronous modulo-9 counter

Step 4: Obtain the minimal expressions: The K-maps for excitations T4, T3, T2, and T1 in terms of the outputs of the FFs Q4, Q3, Q2, and Q1 their minimization and the minimal expressions for excitation obtained from them are shown in the below figure.

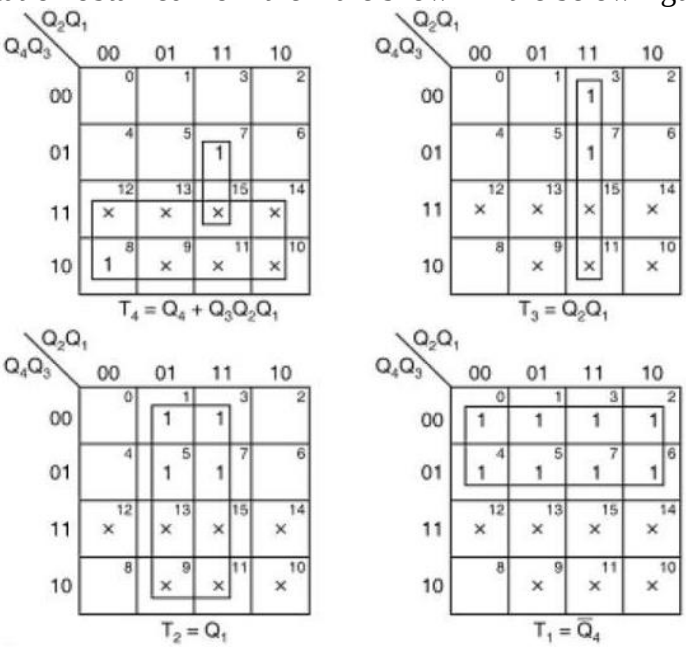


Figure: K-map minimization of synchronous modulo-9 counter using T- flip flops

Step5: Draw the logic diagram: A logic diagram using those minimal expressions can be drawn as shown in the below figure.

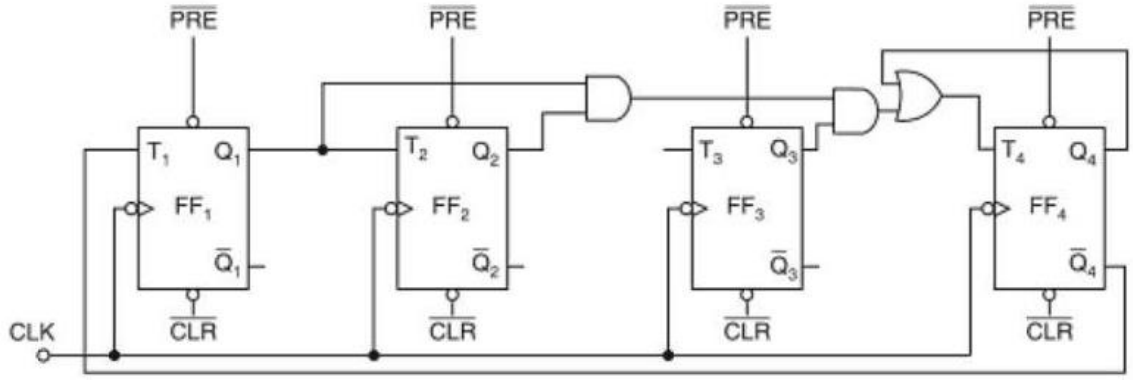


Figure: Logic diagram of synchronous modulo-9 counter using T- flip flops

REGISTERS

An *n*-bit register is a cascade of *n* flip-flops and can store an *n*-bit binary data.

BIDIRECTIONAL SHIFT REGISTER:

A bidirectional shift register is one which the data bits can be shifted from left to right or from right to left. A figure shows the logic diagram of a 4-bit serial-in, serial out, bidirectional shift register. Right/left is the mode signal, **when right /$\overline{\text{left}}$ is a 1**, the logic circuit works as a **shift right-register**. When **right /$\overline{\text{left}}$** is a 0, the logic circuit works as a **shift left-register**.

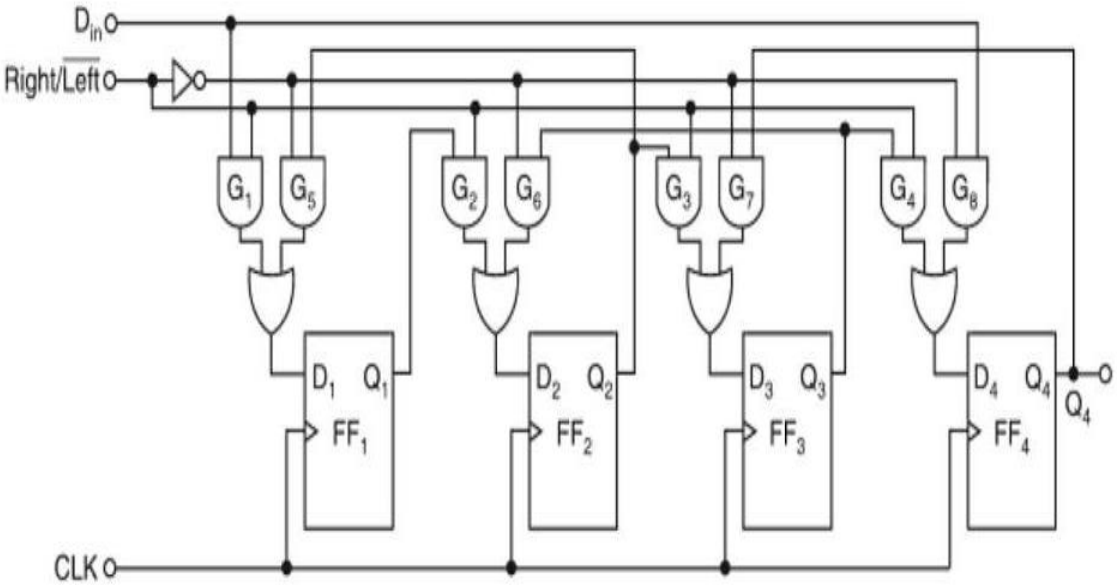


Figure: Logic diagram of 4 bit bidirectional shift register

The bidirectional operation is achieved by using the mode signal and two AND gates and one OR gate for each stage. A **HIGH** on the **right /$\overline{\text{left}}$** control input **enables** the AND gates **G1, G2, G3 and G4** and **disables** the AND gates **G5, G6, G7 and G8**, and the **state of Q output of each FF is passed through the gate to the D input of the following FF.** when a clock pulse occurs**, the data bits** are then effectively **shifted one place to the right**.

A LOW on the **right /$\overline{\text{left}}$** control input enables the AND gates G5, G6, G7 and G8 and disables the AND gates G1, G2, G3 and G4 and the Q output of each FF is passed to the D input of the preceding FF. when a clock pulse occurs, the data bits are then effectively shifted one place to the left. Hence, the circuit works as a bidirectional shift register.

UNIVERSAL SHIFT REGISTER:

A register is capable of shifting in one direction only is a **unidirectional shift register**. One that can shift both directions is **a bidirectional shift register**. If the register has both shifts and parallel load capabilities, it is referred to as a **universal shift registers**. Universal shift register is a bidirectional register, whose **input** can be either in **serial form or in parallel form** and whose **output** also can be in **serial form or in parallel form**.

The most general shift register has the following capabilities.

1. A clear control to clear the register to 0.
2. A clock input to synchronize the operations.
3. A shift-right control to enable the shift-right operation and serial input and output lines associated with the shift-right.
4. A shift-left control to enable the shift-left operation and serial input and output lines associated with the shift-left.
5. A parallel loads control to enable a parallel transfer and the n input lines associated with the parallel transfer.
6. N parallel output lines
7. A control state that leaves the information in the register unchanged in the presence of the clock.

A universal shift register can be realized using multiplexers. The below figure shows the logic diagram of a 4-bit universal shift register that has all capabilities.

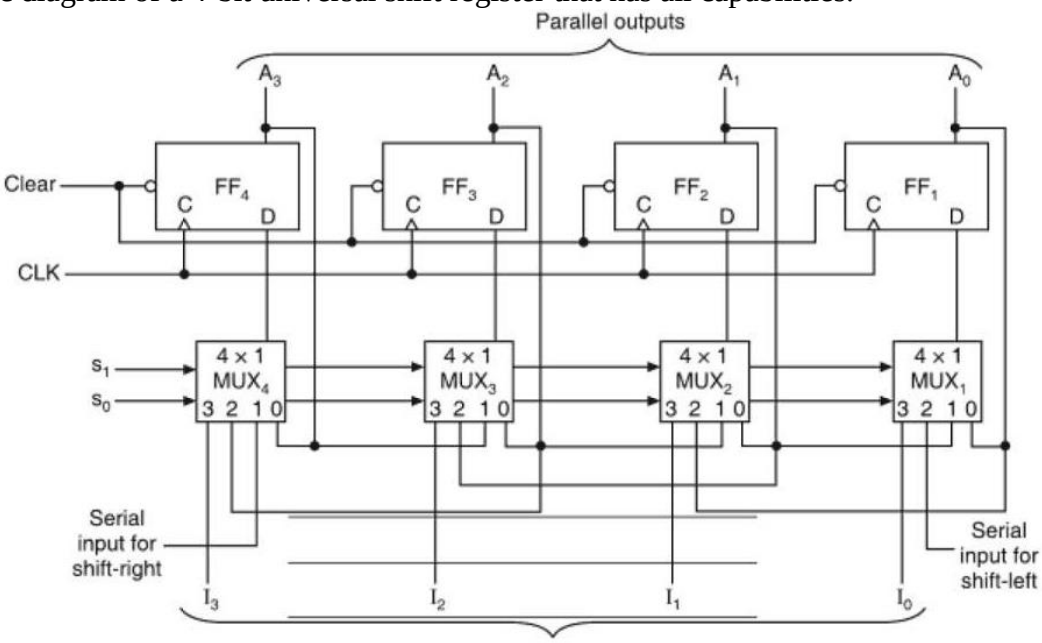


Figure: 4-bit universal shift register

It consists of 4 D flip-flops and four multiplexers. The four multiplexers have two common selection inputs S_1 and S_0. **Input 0** in each multiplexer is selected when **S_1S_0=00**,

input 1 is selected when **S_1S_0=01** and **input 2** is selected when **S_1S_0=10** and **input 3** is selected when **S_1S_0=11**.

The selection inputs control the mode of operation of the register according to the functions entries. **When S_1S_0=0**, the **present value of the register is applied to the D inputs of flip-flops**. The condition forms a path from the output of each flip-flop into the input of the same flip-flop. The next clock edge transfers into each flip-flop the binary value it held previously, and no change of state occurs.

When **S_1S_0=01**, terminal 1 of the multiplexer inputs have a path to the D inputs of the flip-flop. This causes **a shift-right operation**, with serial input transferred into flip-flop4.

When **S_1S_0=10, a shift left operation** results with the other serial input going into flip-flop A1.

Finally, when S_1S_0=11, the binary information on the parallel input lines is transferred into the register simultaneously during the next clock edge.

Mode control		
S_1	S_0	Register operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

Applications of Shift Registers:

1. Time Delays
2. Serial / Parallel data conversion
3. Ring Counters
4. UART (Universal asynchronous receiver transmitter)
5. The shift registers are used for temporary data storage.
6. The shift registers are also used for data transfer and data manipulation.
7. The serial-in serial-out and parallel-in parallel-out shift registers are used to produce time delay to digital circuits.
8. The serial-in parallel-out shift register is used to convert serial data into parallel data thus they are used in communication lines where de-multiplexing of a data line into several parallel line is required.
9. A Parallel in Serial out shift register us used to convert parallel data to serial data.